République Tunisienne

Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

Université de Tunis El Manar

Institut Supérieur d'Informatique d'El Manar

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques

Spécialité : Génie Logiciel et Systèmes d'Information

Par

Sarra Chtouka

Mise en place d'un pipeline d'intégration et de déploiement continus à base de Docker

Encadrant professionnel :	**Monsieur Hamza BEN SALEM**	Ingénieur
Encadrant académique :	**Docteur Ines GAM**	Maître Assistante

Réalisé au sein de Vermeg

Année Universitaire 2016 - 2017

République Tunisienne

Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

Université de Tunis El Manar

Institut Supérieur d'Informatique d'El Manar

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques

Spécialité : Génie Logiciel et Systèmes d'Information

Par

Sarra Chtouka

Mise en place d'un pipeline d'intégration et de déploiement continus à base de Docker

Encadrant professionnel : **Monsieur Hamza BEN SALEM** Ingénieur

Encadrant académique : **Docteur Ines GAM** Maître Assistante

Réalisé au sein de Vermeg

VERMEG

Année Universitaire 2016 - 2017

J'autorise l'étudiante à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Monsieur Hamza BEN SALEM**

Signature et cachet

J'autorise l'étudiante à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrante académique, **Docteur Ines GAM**

Signature

Dédicace

Je dédie ce travail à la prunelle de mes yeux, mes très chers parents Zakaria et Saida, pour tout leur amour, tendresse et sacrifices. Que dieu le tout puissant puisse vous accorder bonheur, longue vie et santé.

A mon frère Alaa-Eddine pour tous les bons moments que nous passons ensemble. Je t'aime.

A mon fiancé Marwen pour sa gentillesse, sa patience et tout le bonheur qu'il ne cesse de m'offrir.

A mes amis de Vermeg, ce fut un plaisir de vous rencontrer.

Remerciement

Au terme de ce stage, je tiens tout d'abord à remercier Vermeg de m'avoir proposé ce stage.

Ensuite, j'exprime ma gratitude envers **Monsieur Hamza BEN SALEM** , qui m'a encadré pendant mon projet de fin d'étude. Je le remercie pour sa disponibilité et l'intérêt qu'il a porté pour ce travail.

Je suis aussi redevable envers Monsieur **Ahmed JABALLAH** pour son aide afin de me permettre de surmonter les difficultés techniques que j'ai rencontré au cours de ce stage.

J'adresse mes chaleureux remerciements à **Docteur Ines GAM** pour ses précieux conseils, son aide et l'effort qu'elle n'a pas ménagé pour m'aider à rédiger ce rapport.

Sarra Chtouka

Table des matières

Introduction générale **1**

1 Cadre général **3**

1.1 Organisme d'accueil 4

1.1.1 Activités et produits de Vermeg 4

1.1.2 Organisation de Vermeg 5

1.1.3 Département d'accueil 5

1.2 Étude de l'existant et constations 6

1.3 Objectifs et contributions 7

1.4 Méthodologie de travail 8

1.4.1 Gestion de projet : Scrum 8

1.4.2 Méthodologie de développement : 2TUP 10

2 Étude préalable **13**

2.1 Intégration continue 14

2.2 Déploiement continu 14

2.3 Éléments clés 15

2.3.1 Docker 15

2.3.2 Cluster 15

2.3.3 Kubernetes 15

2.3.4 Pod 16

2.4 Étude comparative 16

2.4.1 Outils de gestion de versions : Git vs SVN 16

2.4.2 Outils d'orchestration : Jenkins vs Bamboo vs Travis 18

2.4.3 Outil de configuration et de gestion d'ordinateur : Chef vs Puppet vs Ansible 19

2.4.4 Virtualisation : Machine virtuelle vs Docker 20

2.4.5 Outil de gestion de dépôt maven : Nexus sonatype vs JFrog Artifactory . . . 22

2.4.6 Plateforme de déploiement :Openshift vs Cloud foundry 23

2.5 Synthèse et choix 23

3 Analyse et spécifications **25**

3.1 Identification des acteurs . . . 26

3.2 Analyse des besoins . . . 27

3.2.1 Recensement des besoins fonctionnels . . . 27

3.2.2 Rencensement des besoins techniques . . . 28

3.2.3 Besoins non fonctionnels . . . 28

3.3 Représentation des besoins . . . 28

3.3.1 Intégration continue . . . 31

3.3.2 Déploiement continu . . . 35

3.3.3 Configuration . . . 39

3.3.4 Supervision . . . 41

3.3.5 Pré-configuration . . . 44

4 Architecture et conception détaillée **51**

4.1 Architecture générale de la solution . . . 52

4.1.1 Architecture générale basée sur les machines virtuelles . . . 52

4.1.2 Architecture générale basée sur les conteneurs docker . . . 55

4.2 Aspect dynamique de la solution . . . 57

4.2.1 Vue dynamique du volet "Intégration continue" . . . 57

4.2.2 Vue dynamique du volet "Déploiement continu" . . . 58

4.2.3 Vue dynamique du volet "Configuration" . . . 59

4.2.4 Vu dynamique du volet "Pré-configuration" . . . 60

5 Réalisation **70**

5.1 Environnement de travail . . . 71

5.2 Technologies utilisées . . . 72

5.2.1 Technologie de configuration : YAML . . . 72

5.2.2 Technologie de virtualisation : Docker et machine virtuelle . . . 72

5.2.3 Technologie de configuration d'ordinateurs : Ansible . . . 73

5.2.4 Outil de gestion de version : Git . . . 73

5.2.5 Serveur : Apache Tomcat . . . 73

5.2.6 Outil d'orchestration : Jenkins . . . 73

5.2.7 Outil de gestion dépôt maven : Nexus sonatype 73
5.2.8 Plateforme de déploiement : OpenShift 74
5.3 Principaux développements 74
5.3.1 Intégration et déploiement continus 74
5.3.2 Configuration 76
5.3.3 Supervision 77
5.3.4 Pré-configuration 79

Conclusion générale **85**

Bibliographie **87**

Table des figures

1.1 Clients de Vermeg. 4
1.2 Processus actuel. 6
1.3 Chaîne d'intégration continue à mettre en place. 7
1.4 schéma explicatif de la solution proposée. 8
1.5 Scrum. 9
1.6 Processus en Y. 11

2.1 Machine virtuelle vs docker. 21

3.1 Diagramme de cas d'utilisation général. 30
3.2 diagramme de cas d'utilisation "Intégration continue". 32
3.3 Diagramme de cas d'utilisation "Déploiement continu". 36
3.4 Diagramme de cas d'utilisation "Configuration". 39
3.5 Diagramme de cas d'utilisation "Supervision". 42
3.6 Diagramme de cas d'utilisation "Pré-configuration". 44

4.1 Schéma explicatif du pipeline intégration et déploiement continus. 52
4.2 Diagramme de déploiement pour l'architecture à base de machines virtuelles. 54
4.3 Schéma explicatif du pipeline intégration et déploiement continus à base de conteurs Docker. 55
4.4 Diagramme de déploiement pour l'architecture à base de conteneurs Docker 56
4.5 Processus d'Intégration continue 57
4.6 Processus de déploiement d'une application 58
4.7 Processus de création d'un template 60
4.8 Processus d'installation du serveur Git 62
4.9 Processus d'installation du serveur Jenkins 63
4.10 Processus d'installation du serveur Nexus 64
4.11 Processus de supervision de l'état du cluster 66
4.12 Processus pour la pré-configuration. 67
4.13 Processus d'intégration et de déploiement continus dans le pipeline 68

5.1 Job jenkins de type pipeline. 74
5.2 Build effectué sur OpenShift. 75
5.3 Artefacts deployé sur Nexus. 75
5.4 Déploiement d'une application sur OpenShift. 76
5.5 ICréation d'un template OpenShift avec Jenkins job. 77
5.6 Création d'un template OpenShift avec Jenkins job. 77
5.7 Affichage de métriques. 78
5.8 Affichage de l'état des conteneurs. 79
5.9 Logs Ansible. 80
5.10 Console OpenShift. 80
5.11 Extrait de la description de l'état du cluster OpenShift. 81
5.12 Liste des projets Jenkins. 82
5.13 Interface d'authentification Nexus. 83
5.14 Pipeline intégration continue sur OpenShift 84

Liste des tableaux

1.1 Backlog produit 9

2.1 Tableau comparatif Git vs SVN 17
2.2 Tableau comparatif Jenkins vs Bamboo vs Travis 19
2.3 Tableau comparatif Chef vs Puppet vs Ansible 20
2.4 Tableau comparatif machine virtuelle vs Docker 21
2.5 Tableau comparatif Nexus sonatype vs JFrog Artifactory 22
2.6 Tableau comparatif OpenShift Origin vs Cloud Foundry 23
2.7 Tableau récapitulatif des choix technologiques 24

3.1 Description textuelle du cas d'utilisation « Déposer code » 32
3.2 Description textuelle du cas d'utilisation «Récupérer code» 33
3.3 Description textuelle des cas d'utilisation «Déclencher un build» 33
3.4 Description textuelle du cas d'utilisation « Déployer artefacts » 35
3.5 Description textuelle du cas d'utilisation « Déployer application » 36
3.6 Description textuelle du cas d'utilisation « Récupérer image » 37
3.7 Description textuelle du cas d'utilisation « Récupérer URL de production » 38
3.8 Description textuelle du cas d'utilisation « Récupérer URL de test » 38
3.9 Description textuelle du cas d'utilisation « Créer Template » 39
3.10 Description textuelle du cas d'utilisation « Créer script pipeline » 40
3.11 Description textuelle du cas d'utilisation «Surveiller état du cluster» 42
3.12 Description textuelle du cas d'utilisation « Surveiller état des conteneurs » 43
3.13 Description textuelle du cas d'utilisation "Mettre en place un cluster OpenShift" . . 44
3.14 Description textuelle du cas d'utilisation "Configurer métriques" 45
3.15 Description textuelle du cas d'utilisation "Installer serveur Git" 47
3.16 Description textuelle du cas d'utilisation "Installer serveur Jenkins" 48
3.17 Description textuelle du cas d'utilisation "Installer serveur Nexus" 49
3.18 Description textuelle du cas d'utilisation « Mettre en place une chaîne d'intégration continue » 50

5.1 Caractéristiques techniques des machines virtuelles 71

Liste des abréviations

- **CD** = **C**ontinuous **D**elivry
- **CI** = **C**ontinuous **I**ntegration
- **CPU** = **C**entral **P**rocessing **U**nit
- **HTTP** = **H**yper**T**ext **T**ransfer **P**rotocol
- **RAM** = **R**andom **A**ccess **M**emory
- **SSH** = **S**ecure **SH**ell
- **TLS** = **T**ransport **L**ayer **S**ecurity
- **URL** = **U**niform **R**esource **L**ocator
- **YAML** = **Y**et **A**nother **M**arkup **L**anguage

Introduction générale

Habituellement la mise en production d'une application est l'étape ultime d'un processus bien élaboré faisant intervenir, assez souvent, des équipes différentes, à savoir l'équipe de développement et celle des tests. Ainsi, le développement, le test et la mise en production sont assez souvent considères comme trois étapes distinctes.

Faire intervenir autant d'équipes peut mener à des conflits tant le but de chaque équipe est différent de l'autre. Quand les développeurs souhaitent innover et faire évoluer les applications, l'équipe production cherche, avant tout, à maintenir la stabilité du système informatique. D'ailleurs, chacun suit ses processus, et travaille avec ses propres outils qui sont rarement communicants. En conséquence, les relations entre ces équipes peuvent être conflictuelles, ces conflits génèrent des retards de livraison, des coûts supplémentaires pour l'entreprise qui n'ont pas été prévu au départ, avec un impact sur le client dont la satisfaction est au centre des préoccupations de l'entreprise.

Il devient donc évident qu'il faut adopter une nouvelle approche qui permet d'unifier le processus de développement et de production afin d'éviter tous les problèmes précédemment cités.

De ce fait, la notion de DevOps est née. Il s'agit d'une approche qui se base sur la synergie entre la partie opérationnelle et la partie production, l'alignement de l'ensemble des équipes du système d'information sur un objectif commun a pour effet de réduire les conflits entre ces différents intervenants, d'éviter les retards dû à la communication entre eux, et d'améliorer, par conséquent, les délais des livrables (Time-To-Market).

C'est dans ce cadre que s'inscrit notre Projet de Fin d'Etude au sein de la société Vermeg. Il s'agit de mettre en place une platefrome DevOps comprenant un pipeline d'intégration continue et déploiement continu accompagné d'un tableau de monitoring du système mis en place.

Notre rapport s'articule autour de cinq chapitres. Nous présentons, d'abord, un premier chapitre, intitulé "Cadre Général" où nous présentons l'organisme d'accueil, nous exposons notre étude de l'existant et constatations , nous enchaînons avec l'objectif du projet et notre contribution,

pour finir avec la méthodologie de développement adoptée.

Notre deuxième chapitre intitulé "Étude Préalable" permet de clarifier les concepts de base dans notre projet et de procéder par la suite à la comparaison entre les technologies les plus utilisés sur le marché et susceptibles de répondre à notre besoin ensuite nous procédons à la synthèse de nos choix.

Au cours de notre troisième chapitre qui est "Analyse et Spécification" nous identifions les acteurs de notre système, nous spécifions les besoins fonctionnels, techniques et non fonctionnels décelés. Puis, nous procédons à la représentation de ces besoins.

Le quatrième chapitre pour le présent rapport est la "Architecture et conception Détaillée" qui présente l'architecture générale de notre solution ainsi que sa vue dynamique.

Le chapitre "Réalisation" du système mis en place qui permet de présenter l'environnement matériel et logiciel de notre travail, de présenter les différentes technologies que nous avons utilisé pour aboutir à notre travail et pour finir, d'exposer quelques interfaces graphiques relatives à notre travail. Nous clôturons notre rapport par une conclusion générale.

Chapitre 1

Cadre général

Plan

1 Organisme d'accueil 4

2 Étude de l'existant et constations 6

3 Objectifs et contributions 7

4 Méthodologie de travail 8

Introduction

Ce chapitre a pour objectif de situer notre travail par rapport à son cadre général. Nous commençons tout d'abord par la présentation de l'organisme d'accueil, ensuite nous passons à l'étude l'existant et constations. Nous enchaînons avec l'énonciation de l'objectif du projet avant de clôturer par la méthodologie de développement.

1.1 Organisme d'accueil

Fondée en 2002, Vermeg est une entreprise off-shore comptant plus de 700 collaborateurs, et est indépendante de la société mère BFI établie en 1994.
Vermeg dispose d'une expertise en monétique et en édition de logiciels bancaires et financiers, et offre une large gamme de logiciels pour les différents tiers des institutions financières concernés par le traitement des titres. Les produits que Vermeg offre, sont utilisés dans dix-huit différents pays à travers trois continents,la figure 1.1 représente la dispértion géographique de ses différents clients.[1]



Figure 1.1: Clients de Vermeg.

1.1.1 Activités et produits de Vermeg

L'activité de Vermeg s'articule principalement autour de quatre axes :

- Assurance : couverture d'assurance individuelle ou de groupe pour l'épargne et la santé.
- Gestion de richesse et d'actifs : gestion de position et des décisions financières dédiée aux besoins spécifiques de gestion de patrimoine.
- Marché financier et services de sécurité.
- Services financier digitaux : numérisation des processus financier et analyse des données sensibles.

Vermeg fournit pour ses clients un ensemble de logiciels spécialisés dans le domaine financier.[1]

— MEGARA (Securities processing) : La suite MEGARA est une plateforme modulaire pour le traitement des titres destinée essentiellement aux institutions financières qui propose un nombre de modules pouvant être implémentés séparément.

— PALMYRA : est un framework JEE compatible SOA (architecture orientée service) qui incorpore des composants ainsi que des services web réutilisables. L'implémentation de services web réutilisables permet de développer des logiciels de meilleure qualité en temps réduit dont l'architecture est basée sur un client léger.

— SOLIFE : Une solution d'administration de polices d'assurance vie et un portail web à destination des clients finaux/brokers.

— SOLIAM : est une solution de gestion de portefeuilles pour les gestionnaires d'actifs institutionnels de fortune. Ayant une présence internationale en Belgique, France, Irlande, Luxembourg, Pays-Bas, Suisse, Royaume-Uni, Tunisie et des clients dans plus de 23 pays, VermegLife et Soliam sont considérés comme les deux produits phares de Vermeg BSB.

1.1.2 Organisation de Vermeg

Vermeg est compose de quatre unités appelé Business Unit (BU).

— Pension and Insurance : c'est l'unité chargé de développer la solution SoLife pour la gestion des polices d'assurance.

— Wealth and Asset Management : C'est l'unité chargé de développer la solution SOLIAM pour la gestion des portefeuilles des gestionnaires d'actifs institutionnels de fortune.

— Financial Markets and Securities Services : il s'agit de l'unité qui développe la solution Megara dédiée au traitement des titres.

— Digital Financial Services : unité responsable de la solution Palmyra.[1]

1.1.3 Département d'accueil

Durant notre stage, nous avons été affectée au département « Pension and Insurance ». Les efforts de cette équipe sont dédiés aux compagnies d'assurance.

Le département « Pension and Insurance » offre un logiciel d'administration nommé Solife (Vermeg life) qui permet gestion du cycle de vie d'une police d'assurance vie, depuis la soumission de l'offre jusqu'à à la clôture de la police créée.

1.2 Étude de l'existant et constations

Dans cette partie, nous présentons tout d'abord la procédure que Vermeg suivait initialement pour le déploiement ses applications, ensuite nous exposons la problématique.

Pour gérer le cycle de vie de ses applications avant leur déploiement, Vermeg disposait d'une chaîne d'intégration continue basé sur l'outil de gestion de versions "SVN", un serveur d'intégration continue "Jenkins", un gestionnaire de dépôt maven "Nexus sonatype" et enfin pour la mesure de qualité du code source "SonarQube". Une fois les artéfacts de l'application déployée sur Nexus, le client devra télécharger un fichier .war, ensuite il pourra procéder à l'installation de l'application sur son serveur. le figure 1.2 illustre le processus actuel de la chaîne d'intégration chez Vermeg.

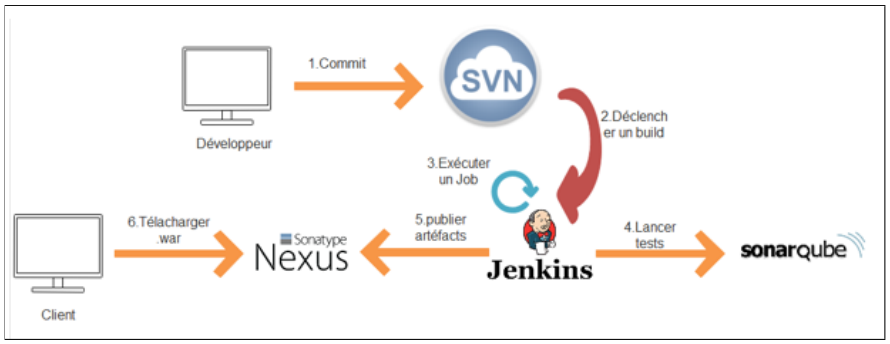


Figure 1.2: Processus actuel.

Bien que Vermeg dispose déjà d'une chaîne d'intégration continue, cette dernière reste insuffisante par rapport à leurs besoins.

Etant donné que SVN est un gestionnaire de version centralisé, il pose le problème de « single point of failure », c'est-à-dire si le répertoire central est endommagé, alors uniquement les données enregistrées dans le dernier backup sont récupérables. En plus, du fait de sa centralisation, SVN rencontre quelques problèmes de performance puisque toutes les opérations doivent passer par le réseau.

Quant au stockage des données, le répertoire de travail contient deux copies de chaque fichier, un pour travailler avec et un pour stocker les métadonnés des fichiers sous format d'un dossier caché avec l'extension.svn tout ceci rend l'espace nécessaire relativement grand.

Le deuxième constat décelé dans le procédé initialement mis en place, est la taille des fichiers artefacts exposé par le serveur Nexus. Les fichiers sont de taille supérieur à 700 Méga Octets, leur téléchargement par les clients prend généralement quelques heures. Il faudra, ensuite, installer l'environnement nécessaire pour le bon fonctionnement de l'application.

L'acheminement de ce processus peut prendre des heures voir quelques jours, un temps qui peut être considérablement réduit si de bonnes pratiques sont appliquées.
Le dernier constat, est la visibilité générale de l'état des systèmes. Actuellement, il n'existe aucun moyen pour Vermeg pour s'assurer que les différents systèmes qui intervenant dans le développement et le déploiement des applications sont fonctionnels et en « bonne santé ».

1.3 Objectifs et contributions

L'objectif de notre projet consiste en la mise en place d'un pipeline d'intégration et de déploiement continus en remplacement à la chaîne initiallement mise en place, ainsi que le développement d'un tableau de bord de monitoring affichant les métriques de performances du pipeline précédemment mise en place. Le pipeline d'intégration continu englobe la mise en place d'un serveur de gestion de version "Git", d'un serveur d'orchestration "Jenkins" et d'un serveur de gestionnaire de dépôt maven "Nexus sonatype". La figure 1.3 illustre le pipeline d'intégration continue que nous voulons mettre en place.

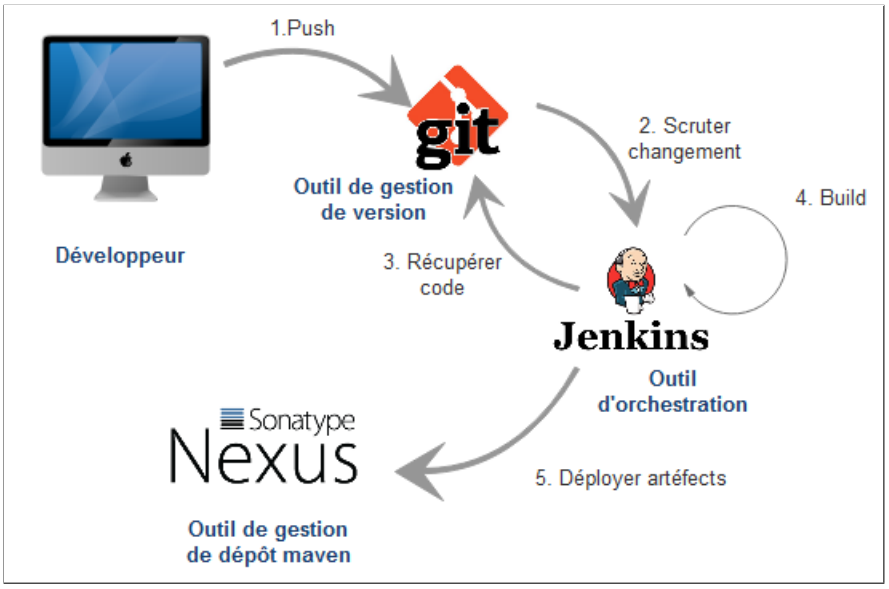


Figure 1.3: Chaîne d'intégration continue à mettre en place.

Bien que le pipeline d'intégration continue existe déjà, les avantages que présente Git par rapport à SVN (voir chapitre étude préalable), font qu'un changement au niveau de ce pipeline est nécessaire.
Avec l'intégration continue nous souhaitons que les équipes de détectent les problèmes le plus tôt

possible, c'est-à-dire, dès que le code soit écrit et non pas à quelques jour de la livraison.

De plus, avec le déploiement continu nous souhaitons déployer automatiquement tout changement effectué qui a réussi une batterie de tests préalablement mis en place.

Afin de mettre en place ce pipeline répondant aux deux derniers points, nous avons eu recourt aux scripts Ansible, qui est un outil de gestion d'ordinateurs à distance, pour déployer un cluster Openshift origin qui sera notre plateforme de déploiement continu.

Grâce à ce pipeline, les clients Vermeg n'ont plus à télécharger des artéfacts et à installer des applications, ils ont simplement accès à une URL, réduisant le temps d'attente d'un client de quelques heures à quelques minutes.

Une fois ces deux pipelines (intégration continue et déploiement continu) sont mis en place et intégrés entre eux, un tableau de bord permettant le monitoring et le contôle des systèmes sera affiché selon les besoins. La figure 1.4 récapitule ce que nous souhaîtons mettre en oeuvre.

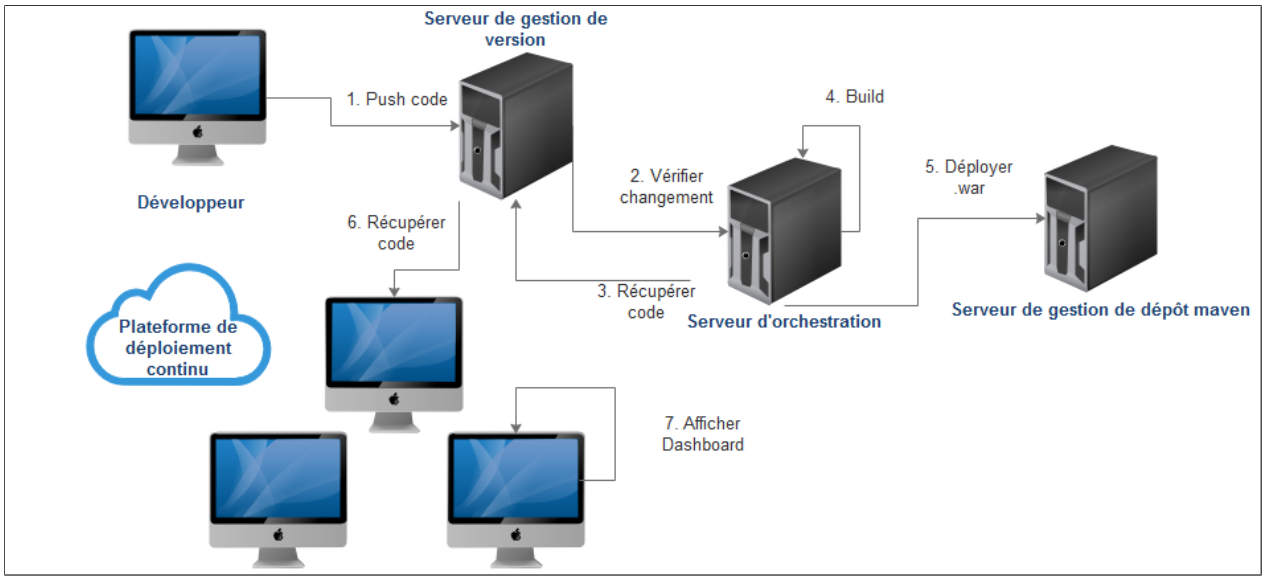


Figure 1.4: schéma explicatif de la solution proposée.

1.4 Méthodologie de travail

Dans cette section, nous allons exposer la méthodologie de travaille utilisée pour la gestion et réalisation de notre projet.

1.4.1 Gestion de projet : Scrum

Pour la gestion de notre projet nous avons suivi un modèle inspiré de Scrum sans l'appliquer pour toutes ses facettes.

Scrum est une méthode agile utilisée pour la « gestion de projet ». Cette méthode de gestion a pour objectif d'améliorer la productivité de l'équipe.

La figure 1.5, illustre le fonctionnement de Scrum, l'équipe se réunit quotidiennement pour faire le point sur ce qui a été fait et à faire. Chacun de nos sprint a duré une semaines. Chaque sprint a produit un livrable.[2]

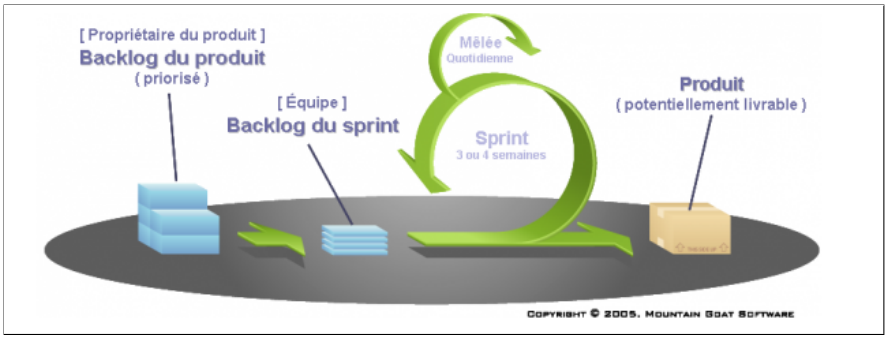


Figure 1.5: Scrum.

Le backlog de produit consiste à énoncer la liste des spécifications fonctionnelles exigés par le client, et est récapitulé dans le tableau 1.1.

L'échelle d'évaluation de la difficulté contient les valeurs : facile, moyen, assez difficile et difficile.

- Facile : est donné pour les tâche les plus basique.
- Moyen : est donné pour les tâches qui incluent une certaine complexité mais qui restent approchable.
- Assez difficile : est donné pour les tâches dont la complexité est assez importante.
- Difficile : est donné pour les tâches dont la complexité est importante.

Tableau 1.1: Backlog produit

Numéro	User story	Description	Priorité	Difficulté	Durée
1	Mettre en place un cluster OpenShift.	En tant que configurateur je peux mettre en place un cluster OpenShift.	2	Assez difficile	4 semaines.

2	Mettre en place une chaîne d'intégration continue.	En tant que configurateur je peux mettre en place une chaîne d'intégration continue	1	Moyen	4 semaines.
3	Configurer projet et le soumettre à une chaîne de déploiement continu .	En tant que développeur je peux créer les configurations nécessaires pour chaque projet et le déployer un projet sur une plateforme de déploiement continu.	3	Difficile	4 semaines.
4	Superviser état du cluster et des conteneurs.	En tant que chef de projet je peux superviser la RAM, le CPU et la pressions des entrées/sorties sur les pods, je peux aussi surveiller l'état des conteneurs (Running, pending,etc.).	4	Assez difficile	4 semaines.

1.4.2 Méthodologie de développement : 2TUP

Pour la réalisation de notre projet, la méthodologie en Y ou 2TUP nous a semblé la plus approprié. La méthodologie 2TUP est un processus de développement unifié, et centré sur l'architecture, son principe fondateur est que toute évolution imposée à un logiciel peut se décomposer et se traiter de façon parallèle, suivant un axe fonctionnel et un axe technique. La réalisation du logiciel consiste en la fusion des résultats de ces deux branches.[3]

La figure 1.6 illustre le processus 2 TUP ou en Y. [4]

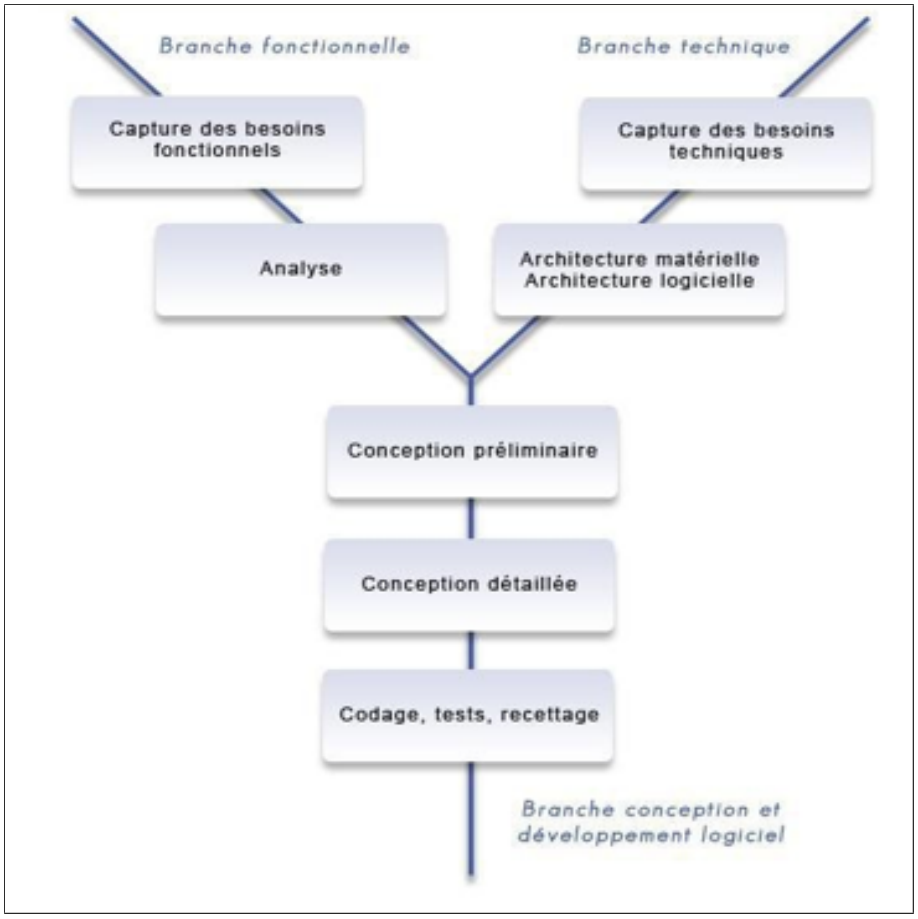


Figure 1.6: Processus en Y.

— Branche technique :Cette branche correspond à la partie où les besoins techniques sont fixés et où l'architecture matérielle et logicielle est énoncé.Au cours de cette partie, nous avons établis, les prérequis matériels tel que les caractéristiques techniques des serveurs à utiliser et les logiciels avec lesquelles nous allons mettre en place nos pipelines.

— Branche fonctionnelle : Cette branche permet la capture des besoins fonctionnels ainsi que leur analyse. Au cours de cette phase, il faut déceler les exigences auxquelles notre système doit répondre. Nous avons donc commencé par faire des recherches sur l'existant, qui nous ont permis de mieux comprendre et de déceler ses anomalies d'un point de vue purement fonctionnel. Nous avons aussi pu établir les besoins fonctionnels et non fonctionnels de notre futur système afin d'éviter tout risque de production d'un système inadéquat.

— Branche conception et développement logiciel : cette branche permet d'intégrer les deux branches précédemment expliquées afin d'aboutir au système final qui correspond à l'objectif de notre

projet.cette phase nous a permis de fusionner les deux branches technique et fonctionnel, ce qui nous a mené à la mise en place d'un pipeline d'intégration et de déploiement continus comprenant un système de monitoring sur l'état des différentes composantes qui interviennent dans le pipeline.

Pour la documentation de ces volets, nous avons choisi de préciser toutes les exigences des différented natures dans chaque description de cas d'utilisation.

Conclusion

Dans cette partie de notre rapport, nous avons présenté l'organisme d'accueil, ensuite nous sommes passée à l'étude de l'existant et constatation. Nous avons aussi énoncé l'objectif de notre projet avant de clôturer avec Méthodologie de développement adoptée.

Chapitre 2

Étude préalable

Plan

1 Intégration continue 14

2 Déploiement continu 14

3 Éléments clés 15

4 Étude comparative 16

5 Synthèse et choix 23

Introduction

Dans ce chapitre, nous commençons par définir l'intégration continue et le déploiement continu, ensuite, nous exposons quelques éléments clés avant de procéder à une étude comparative. Enfin, nous faisons la synthèse de nos choix technologiques.

2.1 Intégration continue

L'intégration continue est définit comme étant un ensemble de pratiques utilisés afin de s'assurer que chaque modification dans le code source ne produit pas des problèmes de régression dans l'application en cours de développement, c'est-à-dire qu'aucun défaut n'a été introduit à la partie du système qui n'a pas été modifié.[5]
Le principal but de cette approche est d'anticiper et d'identifier rapidement les bugs tôt avant la mise en production du logiciel. Cela permet d'avoir une vision plus complète du logiciel, notamment sur les différents points faibles et points forts du code ou de l'équipe. Ceci permet de gagner en réactivité pour faire face aux différents problèmes pouvant être présents dans les diverses phases du projet.
Pour bénéficier des avantages qu'apporte le concept d'intégration continue, il est nécessaire de suivre quelques règles de bonnes pratique comme le maintien d'un unique dépôt pour le code source visionné, effectuer plusieurs commit par jour et par développeur, automatiser la compilation du code, et le maintien d'un de compilation court.
L'architecture d'un pipeline d'intégration continu, fait intervenir plusieurs éléments :

— Un gestionnaire de code.

— Un serveur d'orchestration ou d'intégration continu.

— Un serveur de gestion de dépôt.

2.2 Déploiement continu

Le déploiement continu est le processus où toute modification du code, comprenant des tests automatisés et autres vérifications appropriées, est automatiquement déployée en production. L'objectif est de réduire la durée du cycle ainsi que le temps et l'effort du processus de déploiement. Cela permet également aux équipes de développement de réduire le temps nécessaire pour la livraison de fonctionnalités individuelles ou des corrections de bogue, et par conséquent augmente considérablement

la productivité des équipes. Eliminer ou réduire ces périodes d'intenses activités menant à une livraison traditionnelle ou à un déploiement libère également des ressources et du temps pour améliorer les processus. [6]

2.3 Éléments clés

Dans cette section, nous définissons quelques éléments clés, à savoir docker, cluster, kubernetes et pod.

2.3.1 Docker

Docker est un programme Open source qui permet l'intégration d'une application open-source et de ses dépendances sous la forme d'un conteneur linux. Il se base sur le principe de la virtualisation par le biais de conteneur ce qui permet d'isoler les applications les unes des autres sur un même système d'information. Ces conteneurs ont l'avantage d'être portables sur les différentes distributions Linux à savoir Fedora, Ubuntu, OpenSUSE, CentOs, Mageia, RedHat, etc.
Un conteneur Docker contient ce dont une application a besoin (fichiers binaires, librairies et système de fichier). [7]

2.3.2 Cluster

Dans un système informatique, un cluster, est définit comme étant un groupe de ressources, telles que des serveurs ou des ordinateurs. Ce groupe se comporte comme un seul et même système. Il en résulte ainsi une disponibilité élevée, et parfois, des fonctions de traitement parallèle et d'équilibrage de la charge. C'est la notion de clustering. [8]

2.3.3 Kubernetes

Kubernetes est un programme open source dédié à la gestion de conteneurs Linux. Il automatise le déploiement, l'évolution, la maintenance, la planification et le fonctionnement de nombreux conteneurs d'application sur des clusters. Il contient des outils d'orchestration, de catalogages de services et équilibrage de charge utilisables avec les conteneurs Docker. Kubernetes exécute les conteneurs dans des pods. [9]

2.3.4 Pod

Un pod est l'unité qui héberge un ou plusieurs conteneurs. Ces derniers, partagent des ressources. Pour chaque pod, Kubernetes choisit une machine ayant une capacité de traitement suffisante et lance les conteneurs associés. [9]

2.4 Étude comparative

Pour la mise en place d'un pipeline d'intégration et de déploiement continus et d'un tableau de monitoring qui relate l'état du système, nous avons besoin d'un ensemble d'outils à savoir, un gestionnaire de version, un orchestrateur, un serveur de dépôt maven, un serveur de déploiement continu. De plus, pour faciliter la tâche de la mise en place de certains de ces éléments, nous avons aussi besoin d'un outil de configuration et de gestion d'ordinateur.
Dans cette section du rapport nous procédons à une étude comparative entre les différents outils existant sur le marché et pouvant répondre à notre besoin.

2.4.1 Outils de gestion de versions : Git vs SVN

Git et SVN sont les deux outils de gestion de version les plus répandu sur le marché, voici un tableau comparatif entre ces deux outils qui nous permettra par la suite d'effectuer notre choix technique. Le tableau 2.1 compare entre les deux outils de gestion de versions Git et SVN en se basant sur les éléments suivant : [10]

— architecture,

— single point of failure,

— contrôle d'accès,

— stockage du contenu,

— branches,

— espace mémoire utilisé.

Tableau 2.1: Tableau comparatif Git vs SVN

Aspect	**SVN**	**Git**
Architecture	Centralisé, uniquement le répertoire central a l'historique complet des changements. Les utilisateurs doivent communiquer via le réseau avec le répertoire central pour obtenir l'historique. Les backup sont gérés et maintenu indépendamment du SVN.	Distribué, tous les développeurs qui vérifient le code à partir d'un dépôt / serveur central auront leur propre référentiel cloné installé sur leur machine. Ce qui permet les développeurs de pouvoir créer une nouvelle branche, de faire un « commit » sur un fichier et de revoir une version même en absence de connexion.
Single point of failure	Si le répertoire central est endommagé, alors uniquement les données enregistrées dans le dernier backup sont récupérable.	La notion de « single point of failure » n'existe pas car il y a autant de copie du dossier qu'il y'en a d'utilisateur si ce n'est plus.
Contrôle d'accès	Nécessité d'un « commit access » due au fait de la centralisation.	Non nécessite d'un « Commit access » puisque le système est distribué. Il faut juste décider de fusionner quoi à partir de quel utilisateur.
Stockage de contenu	Stockage des métadonnées des fichiers sous forme de dossier caché .svn	Stockage de tout le contenu dans un dossier .git , c'est le dépôt du code cloné sur la machine client.

Les branches	Les branches sont gérées comme étant un autre dossier dans le dépôt du code, elles ne sont pas historiées. Pour connaître s'il y a eu fusion entre des branches, il faut explicitement exécuter une commande. Cette façon de gérer les branches démultiplie les chances de créer des branches « orpheline». Une fois la fusion est faite on ne peut plus savoir ce qui a été modifié.	La gestion des branches est assez simple, on peut facilement naviguer entre les différentes branches d'un même répertoire de travail. Chaque répertoire de travail existant chez un utilisateur est considéré comme une branche. La fusion de la branche inclut les informations suivantes : auteur, heure, date, informations relatives à la branche et à la révision, les changements effectués par rapport à la version antécédente. la fusion suivante, commence automatiquement là où la dernière fusion a été faite.
Espace	Le répertoire de travail contient deux copie de chaque fichier, un pour travailler avec et un caché dans le .svn, ce qui rend l'espace nécessaire relativement grand.	Taille du dépôt de code et répertoire de travail relativement petit (30X inférieur à celui de SVN). Ceci est due au fait que pour Git, le répertoire de travail nécessite seulement un fichier d'index de taille 100 bits pour chaque fichier.

Suite à cette comparaison, nous avons opté pour le serveur Git, qui grâce à son architecture décentralisé et le gain en espace qu'il fournit (dépôt 30 fois inférieur à celui de SVN) répond au mieux aux nouvelles attentes de Vermeg.

2.4.2 Outils d'orchestration : Jenkins vs Bamboo vs Travis

Jenkins, travis et bamboo sont des outils d'orchestration bien connu sur le marché, chacun d'entre eux présente ses avantages et inconvénients. Le tableau 2.2 permet de comparer entre ces

outils, la comparaison se base sur :

— l'aspect Open source du code,

— le nombre de plugins,

— la simplicité d'utilisation,

— l'interfaçage avec les différents outils de gestion de version.

Tableau 2.2: Tableau comparatif Jenkins vs Bamboo vs Travis

Aspect	**Jenins**	**Bamboo**	**Travis**
Open source	Oui	Non	Oui
Plugins	Oui	Cher	Peu
Simplicité	Non	Oui	Non
Outil de versionning	Tout	Tout	Github

L'étude comparative entre ces différents serveurs d'orchestration a dirigé notre choix vers Jenkins, qui grâce à ses nombreux plugins, permet un interfaçage facile avec de nombreux outils dont nous avons besoin tel que Git et OpenShift.

2.4.3 Outil de configuration et de gestion d'ordinateur : Chef vs Puppet vs Ansible

La mise en place des serveurs peut s'avérer une tâche fastidieuse répétitive et sujette aux erreurs de manipulation. L'apparition de "Chef", "Puppet" et "Ansible" a été longuement attendu. Grâce à ces outils il n'est plus nécessaire de configurer chaque poste individuellement, il suffit de lancer un script, et des dizaines, centaines ou voir millier de machines sont configuré toutes en mêmes temps, réduisant ainsi l'effort fourni et les risques d'erreur du à la manipulation.Ici nous dressons un tableau comparatif entre ces trois outils de configuration basé sur les aspects suivant : [11]

— le code (open source ou non),

— l'infrastructure,

— le langage,

— l'architecture de chaque outil.

Tableau 2.3: Tableau comparatif Chef vs Puppet vs Ansible

Aspect	**Chef**	**Puppet**	**Ansible**
Open source	Oui	Oui	Oui
Infrastructure	Mutable	Mutable	Mutable
Langage	Procédural	Déclaratif	Procédural
Architecture	Client/serveur	Client/serveur	Client uniquement

Le fait qu'Ansible ne nécessite aucune installation coté client nous a incité à le choisir en tant que serveur de configuration d'ordinateur.

2.4.4 Virtualisation : Machine virtuelle vs Docker

Les machines virtuelles reposent sur le principe de la virtualisation du matériel, c'est-à-dire la création d'un environnement virtuel complet simulant un ordinateur avec du « faux matériel » le système d'exploitation invité communique uniquement avec ce « faux matériel » ce qui rend l'environnement virtualisé étanche. Une couche hyperviseur est ajoutée par-dessus du système d'exploitation pour permettre aux différents systèmes d'exploitation de travailler sur une même machine physique. Docker utilise Docker Engine par-dessus du système d'exploitation hôte qui permet la virtualisation du kernel, Docker utilise le même Kernel et le même réseau que sa machine hôte d'où sa description comme étant « léger». La figure 2.1, ci-dessus illustre la différence architecturale entre une machine virtuelle et Docker. [12]

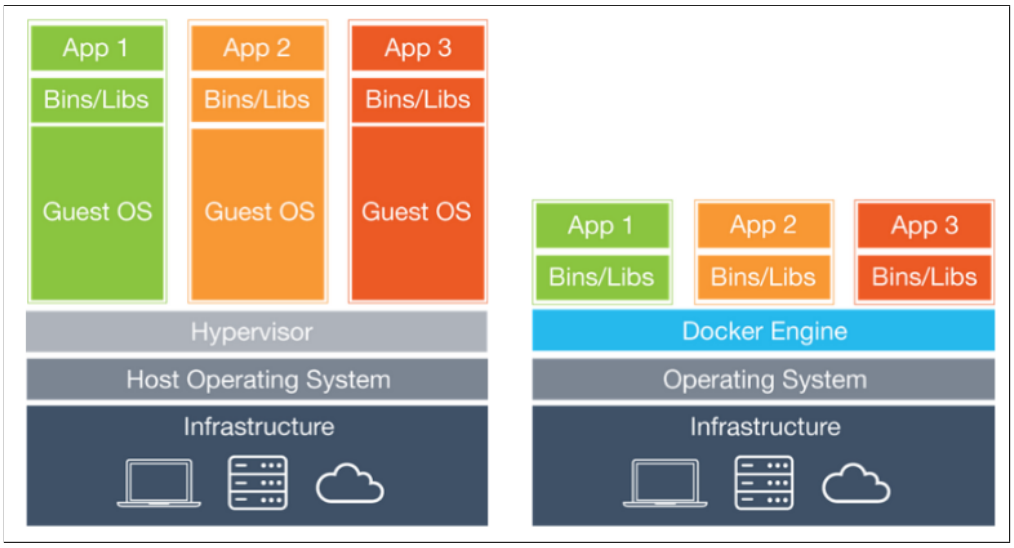


Figure 2.1: Machine virtuelle vs docker.

Le tableau 2.4, ci-dessous illustre une comparaison entre la machine virtuelle et Docker basé sur :

- la virtualisation,
- la plateforme de virtualisation,
- le système d'exploitation hôte,
- l'overhead,
- le chargement du système d'exploitation,
- l'isolation,
- le démarrage,
- le nombre d'instances.

Tableau 2.4: Tableau comparatif machine virtuelle vs Docker

Aspect	**Machine virtuelle**	**Docker**
virtualisation	Hardware	Kernel
Plateforme de virtualisation	hyperviseur	Docker Engine
Système d'exploitation hôte	Tout (distribution linux, windows , mac)	Distribution linux
Overhead	Oui	moins important

Chargement système d'exploitation	Son propre système d'exploitation	Aucun
Isolation	Oui	moins importante
démarrage	Prend quelques minutes	Rapide
Nombre d'instances	Quelques-unes	important

A l'issue de cette comparaison nous déduisons que nous avons besoin de combiner les deux technologies pour répondre à nos besoins.

2.4.5 Outil de gestion de dépôt maven : Nexus sonatype vs JFrog Artifactory

L'utilisation d'un gestionnaire de dépôt maven a plusieurs utilités, le dépôt peut servir d'un backup sur les applications dont les artefacts sont déployés sur ces serveurs, les artefacts peuvent être utilisé comme des bibliothèques et peuvent être importé par les différents développeurs sur d'autres applications.

Nexus et JFrog Artefactory sont les deux gestionnaires de dépôts maven les plus répandu sur le marché. Le tableau 2.5 est un tableau comparatif sur les deux outils JFog Artifactory et Nexus sonatype, cette comparaison est basé sur : [13]

— Le code (Open source ou non).

— Installation à base de Docker.

— Dockerfile.

— Taille du jar.

Tableau 2.5: Tableau comparatif Nexus sonatype vs JFrog Artifactory

Aspect	**JFrog Artifactory**	**Nexus sonatype**
Open source	Oui	Oui
Installation à base de Docker	Moyen	Facile
Dockerfile	Non disponible	Disponible
Taille du jar	37M	13M

Cette comparaison a été faîtes pour un but académique, le choix du gestionnaire du dépôt maven avec lequel nous allons travailler a été fait par l'entreprise hôte Vermeg., il s'agit de Nexus sonatype.

2.4.6 Plateforme de déploiement :Openshift vs Cloud foundry

OpenShift et Cloud foundry sont tous les deux des solutions PAAS pour le déploiement des applications sur des modèle cloud public ou privé et permettent l'hébergement en cloud computing. Ces outils peuvent être intégrés dans un pipeline de déploiement continu d'où notre intérêt.
Le tableau comparatif 2.6 sur les deux outils OpenShift Origin et Cloud Foundry se base sur les éléments suivant : [14]

— Cloud.

— Outils supporté.

— Systèmes d'exploitation

Tableau 2.6: Tableau comparatif OpenShift Origin vs Cloud Foundry

Aspect	**OpenShift**	**Cloud Foundry**
Cloud	PAAS	PAAS
Outils supporté	Data Dog, Jenkins, Eclipse, Jboss studio.	Github, Eclipse, Spring.io, CloudBees
Systèmes d'exploitation	Windows, Linux, Mac	Windows, Mac

Cette comparaison a été faîtes pour un but académique, le choix de la solution a été fait par l'entreprise hôte Vermeg., il s'agit d'OpenShift Origin.

2.5 Synthèse et choix

À L'issu d'une étape d'étude comparative, nous avons procédé aux choix technologiques énoncé mais que nous allons récapituler.
Comme serveur de gestion de version nous avons opté pour Git, et pour l'outil d'orchestration, notre choix s'est porté sur Jenkins.

Pour la configuration de notre cluster, nous avons utilisé l'outil de configuration d'ordinateur Ansible.
Comme technique de virtualisation, nous avons choisi la combinaison entre virtualisation matérielle avec les machines virtuelles et la virtualisation du Kernel avec Docker.
L'outil de gestion de dépôt maven que nous avons gardé est Nexus sonatype.
Enfin, nous avons choisi OpenShift origin comme plateforme de déploiement continu. Le tableau 2.7 récapitule nos choix.

Tableau 2.7: Tableau récapitulatif des choix technologiques

Fonctionnalité	**Outil**
Serveur de gestion de version	Git
Serveur d'orchestration	Jenkins
Configuration du cluster	Ansible
Outil de gestion de dépôt maven	Nexus sonatype
Plateforme de déploiement continu	OpenShift origin

Conclusion

Au cours de ce chapitre, nous avons définit l'intégration continue et le déploiement continu, nous avons aussi présenté quelques éléments clés. Nous avons, par la suite, fait une étude comparative sur les outils qui peuvent répondre à notre besoin et enfin, nous avons fait la synthèse de nos choix technologique. Le prochain chapitre portera sur l'analyse et spécifications.

Chapitre 3

Analyse et spécifications

Plan

1 Identification des acteurs 26

2 Analyse des besoins 27

3 Représentation des besoins 28

Introduction

Dans le présent chapitre, nous nous intéressons aux fonctionnalités que notre système doit satisfaire. Pour cela nous commençons ce chapitre par identifier les acteurs de notre système. Ensuite, nous procédons à l'analyse des besoins pour clôturer avec la représentation des besoins de notre projet. Cette représentation est faite avec UML. Pour les besoins de ce chapitre, nous utilisons les diagrammes de cas d'utilisation accompagné de description textuelle.

3.1 Identification des acteurs

Pour notre système, nous avons identifié 5 acteurs.

— Développeur : c'est un utilisateur de GIT, il peut déposer ou récupérer un code sur le dépôt, c'est le premier acteur à déclencher le pipeline.

— Client : c'est l'utilisateur final, il devra disposer d'une URL de production qui le fait accéder à l'application.

— Chef de projet : c'est l'utilisateur concerné par le monitoring, il dispose d'un tableau de supervision qui lui permet de connaître l'état du cluster.

— Testeur : c'est un acteur qui intervient dans la phase du test du logiciel, il dispose d'une URL de test pour tester les fonctionnalités de l'application.

— Configurateur : il intervient dans la phase de configuration du système et a pour rôle la mise en place du pipeline.

Notre système inclut aussi certains composants comme étant des acteurs :

— Git : c'est une gestionnaire de versions de code, il permet de stocker le code et de l'exposer aux différents développeurs.

— Jenkins : c'est un orchestrateur et un outil important dans l'intégration continue, il permet de déclencher des builds.

— Nexus : c'est une gestionnaire de dépôt maven, il permet d'exposer des artefacts.

— Docker registry : c'est un composant qui permet le stockage des images docker et leur récupération.

3.2 Analyse des besoins

Dans cette partie, nous commençons par analyser les besoins fondamentaux de notre système à travers la partie besoins fonctionnels, les besoins techniques et pour finir avec les besoins non fonctionnel à considérer pour ce système.

3.2.1 Recensement des besoins fonctionnels

— Le système permet au développeur d'exposer ou récupérer un code sur un serveur de dépôt de code.

— Le système permet au développeur de créer un script pipeline.

— Le système permet au développeur de créer un Template pour le build et le déploiement de l'application sur OpenShift.

— Automatiser le déclenchement d'un build sur Jenkins suite à un dépôt de code sur Git.

— Visionner le build sur OpenShift.

— Récupérer les artefacts générés par Nexus suite à un build réussit.

— Automatiser le déploiement des applications sur OpenShift.

— Afficher au sein d'OpenShift, les différentes étapes par lesquelles passe le code avant d'être déployé.

— Le système permet au client d'accéder à l'application via une URL de production qu'il expose par.

— Le système permet au testeur de tester le bon fonctionnement de l'application en y accédant via l'URL de test qu'il expose.

Le système permet au chef de projet de superviser le pipeline, pour ce faire :

— Le système permet au chef de projet de surveiller l'usage CPU, de la RAM et la pression entrée/sortie sur le réseau par les pods.

— Le chef de projet peut savoir l'état des différentes applications existant sur OpenShift (container creating, running, pending,etc.).

3.2.2 Rencensement des besoins techniques

les besoins techniques sont :

— Six machines virtuelles (Git, Nexus, Jenkins,Ansible,OpenShift master,OpenShift slave).

— Image iso centos 6 ou 7.

— Image iso atomic centos 7.

— Docker 1.6.2 ou plus

— Ordinateur doté d'une RAM 12GB au minimum.

3.2.3 Besoins non fonctionnels

Par conjonction aux besoins fonctionnels précédemment cités, notre système doit respecter un nombre de critères donnant une meilleure qualité de la solution obtenue. Parmi ces critères nous citons :

— Performance : le temps de réponse du système, à savoir le pipeline et le tableau de monitoring, doit être minimal, au clique sur l'URL, le tableau de monitoring doit pouvoir s'afficher instantanément.

— Extensibilité : le système doit pouvoir supporter l'évolution et l'extensibilité de ses composants, nous pouvons ainsi ajouter autant de nœud à notre Cluster OpenShift que notre besoin l'exige.

— Convivialité : le tableau de monitoring doit être compris de façon intuitive, c'est-à-dire les indicateurs de performances doivent être affiché et commenté de façon claire et sans ambiguïté.

3.3 Représentation des besoins

Au niveau de cette partie, nous traduisons les besoins précédemment dégagés, sous forme de diagrammes de cas d'utilisation.

Le diagramme de cas d'utilisation général, nous permet d'exposer de façon semi-formelle les principales fonctionnalités auxquels le système doit répondre.En effet nous avons identifié différents sous système pour le pipeline, à savoir :

— CI : continuous integration ou intégration continue.

— CD : continuous deployement ou déploiement continu.

— Configuration.

— Supervision.

— Pré-configuration.

La figure 3.1 illustre le diagramme de cas d'utilisation général de notre système.

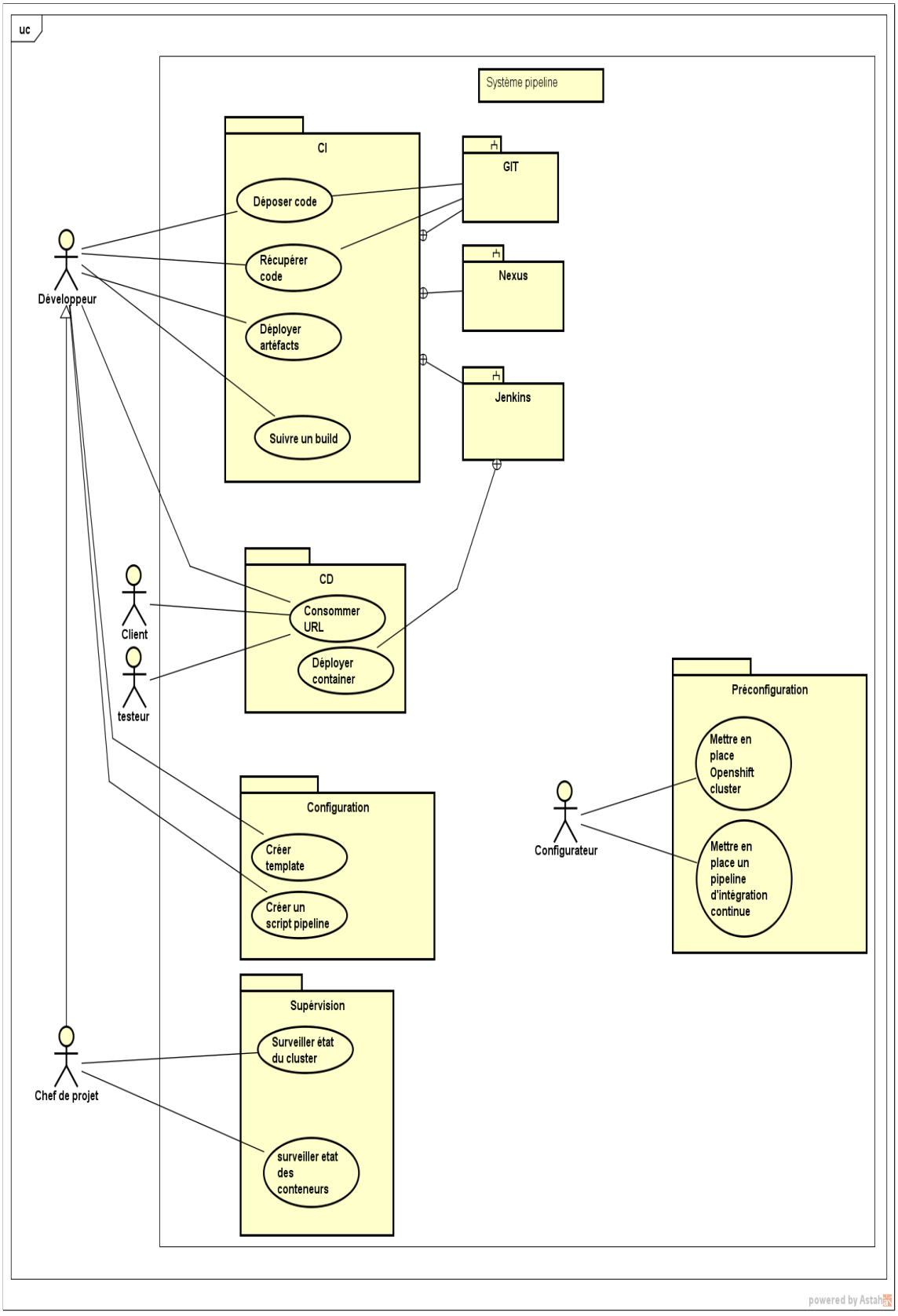


Figure 3.1: Diagramme de cas d'utilisation général.

Le diagramme de cas d'utilisation général a mis en évidence, les principales fonctionnalités du système. Ainsi, nous décelons cinq sous systèmes pour notre projet, le premier etant l'intégration continue, le deuxième c'est le déploiement continu, le troisième est la configuration, le quatrième volet c'est le monitoring et enfin, le dernier volet c'est la pré-configuration qui inclut la mise en place d'un pipeline d'intégration et de déploiement continus, cette étape se fait une seule fois

Le développeur a la possibilité d'agir sur le système avec plusieurs façons. Il peut :

— Déposer ou récupérer un code à partir de GIT.

— Suivre un build sur Jenkins.

— Déployer actefacts sur Nexus.

— Accéder à l'URL de l'application déployée sur OpenShift.

— Créer un Template pour chaque projet qui sera déployé sur OpenShift.

— Créer un script pipeline qui décrit les étapes par lesquelles passe le code (développement, test, déploiement, etc.).

Le chef de projet est un développeur qui, en plus des cas d'utilisation possible pour ce dernier, il peut :

— Surveiller l'état du cluster, à savoir la RAM, le CPU et la préssion des entrée/sortie réseau sur les pods.

— Surveiller l'état des projets, c'est-à-dire, lister les pods et conteneurs existant par projet, et préciser ceux qui sont fonctionnels et ceux qui ne le sont pas.

Les clients et les testeurs peuvent accéder à l'application grâce à une URL qui sera exposé par OpenShift.
Le configurateur peut créer et mettre en place un cluster OpenShift ainsi qu'un pipeline d'intégration continue.

3.3.1 Intégration continue

Le volet d'intégration continue, se traduit en un pipeline qui commence lorsqu'un développeur expose son code sur le serveur de gestion de version Git et prend fin lorsque les artefacts du code sont déployé sur le serveur Nexus. Le diagramme de cas d'utilisation « Intégration continue » représenté par la figure 3.2 explique les fonctionnalités exigés dans cette partie du travail.

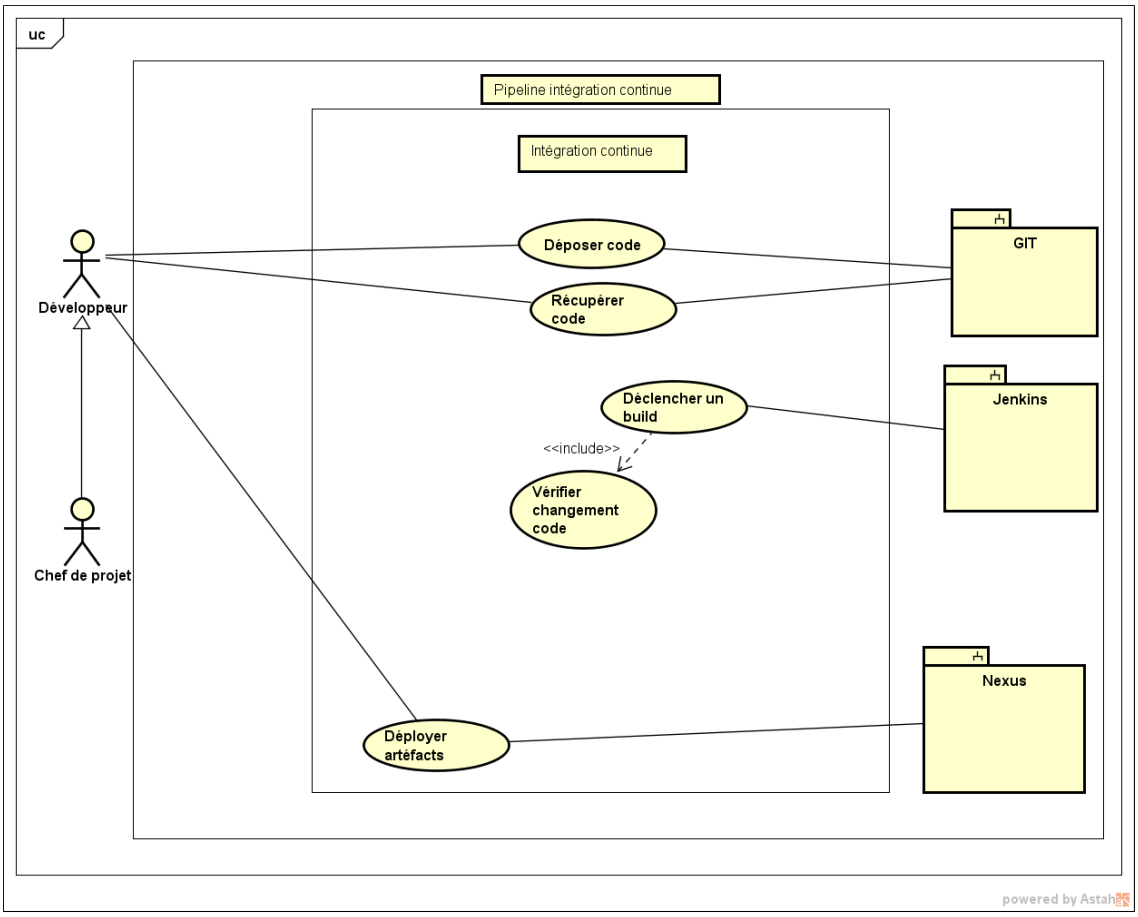


Figure 3.2: diagramme de cas d'utilisation "Intégration continue".

Tableau 3.1: Description textuelle du cas d'utilisation « Déposer code »

Acteur :	Développeur
Précondition :	connexion ssh ou http établie entre le poste utilisateur et le serveur Git.
Post condition :	code déposé sur le serveur Git.
Description :	Le Développeur doit effectuer une opération de « push » afin de déposer le code sur le serveur Git
Scénario nominal :	1. Le développeur effectue un push. 2. Le code est exposé sur le serveur Git.

Scénario Alternatif :	1. Le serveur Git décèle un conflit. 2. Résoudre le conflit manuellement ou automatiquement. Retour au point 1 du scénario nominal.
Scénario d'erreur :	La connexion ssh ou http entre le poste utilisateur et le serveur Git n'est pas établie, le code ne peut pas être déposé sur le serveur.

Tableau 3.2: Description textuelle du cas d'utilisation «Récupérer code»

Acteur :	Développeur
Précondition :	Connexion ssh ou http établi entre le poste utilisateur et le serveur git.
Post condition :	Code est récupéré sur le post du développeur.
Description :	Le Développeur doit effectuer une opération de « pull » afin de récupérer le code à partir serveur Git.
Scénario nominal :	1. le développeur lance une requête « pull » 2. le code est récupéré sur le poste client.
Scénario Alternatif :	Aucun.
Scénario d'erreur :	La connexion ssh ou http entre le post utilisateur et le serveur Git n'est pas établi, le code ne peut pas être récupéré à partir du serveur.

Tableau 3.3: Description textuelle des cas d'utilisation «Déclencher un build»

Acteur :	Jenkins
Précondition :	Connexion ssh ou http établi entre le serveurs Jenkins et Git.
Post condition :	Le build est effectué avec un succès.

Description :	Le serveur Jenkins, vérifie périodiquement si un changement au niveau du code est détecté sur Git, si oui, il déclenche automatiquement un build.
Scénario nominal :	1. Le développeur, définit au sein de Jenkins, un intervalle de vérification du serveur Git. 2. Jenkins vérifie le changement du code au niveau de git une fois le temps définit par l'intervalle est écoulé. 3. Jenkins détecte un changement et déclenche un build. 4. Jenkins affiche un message de succès et crée un artefact.
Scénario Alternatif :	— Jenkins ne détecte aucun changement de code au niveau de l'étape 2 du scénario nominal. Le scénario nominal reprend au point 2 après l'écoulement de l'intervalle de temps définit par le développeur. — Jenkins affiche un message d'erreur à l'étape 4 du scénario nominal.
Scénario d'erreur :	La connexion ssh ou http entre Jenkins et le serveur Git n'est pas établi, la communication entre les deux serveurs ne peut pas se faire.

Tableau 3.4: Description textuelle du cas d'utilisation « Déployer artefacts »

Acteur :	Développeur
Précondition :	Build effectué avec succès, connexion ssh ou http établie entre le serveur Jenkins et Nexus.
Post condition :	Un artefact est déployé sur le serveur Nexus.
Description :	Le fichier "pom.xml" du projet maven, contient l'adresse du serveur Nexus. Une fois le build effectué avec succès, l'artefact généré est déployé sur l'adresse du serveur précédemment précisé.
Scénario nominal :	1. Le développeur précise l'adresse du serveur Nexus dans le fichier pom.xml. 2. L'artefact est déployé sur le serveur Nexus.
Scénario Alternatif :	Aucun
Scénario d'erreur :	— La connexion ssh ou http entre Jenkins et le serveur Git n'est pas établi, la communication entre les deux serveurs ne peut pas se faire. — L'adresse du serveur Nexus dans le fichier pom.xml est incorrecte, l'artefact ne peut pas être déployé sur Nexus.

3.3.2 Déploiement continu

Le module de déploiement continu, se traduit en un pipeline qui inclut le serveur Jenkins, OpenShift et docker registry. Le diagramme de cas d'utilisation « déploiement continu» représenté par la figure 3.3 explique les fonctionnalités offertes par ce volet de notre projet.

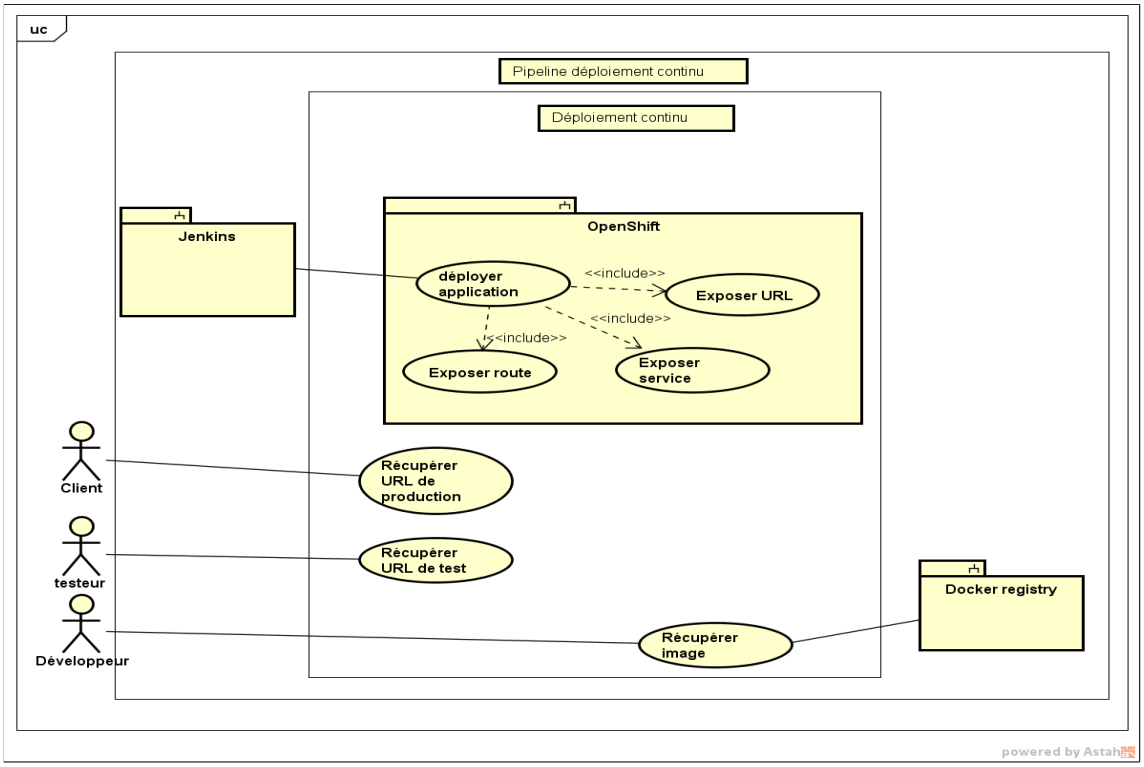


Figure 3.3: Diagramme de cas d'utilisation "Déploiement continu".

Tableau 3.5: Description textuelle du cas d'utilisation « Déployer application »

Acteur :	Jenkins
Précondition :	Build effectué avec succès et une image docker est créé.
Post condition :	Application déployée et accessible via une route.
Description :	L'application est conteneurisée sous format d'un conteneur docker et déployé sur OpenShift en exposant un service qui la rendra accessible via une route.

Scénario nominal :	1. OpenShift crée un pod au niveau d'un nœud du cluster. 2. OpenShift assigne le pod crée à un nœud. 3. OpenShift assigne une adresse ip interne au pod qui sera accessible que par les nœuds du cluster. 4. OpenShift crée le conteneur au sein du pod et ceci à partir de l'image docker précédemment générée. 5. OpenShift expose le service du conteneur. 6. OpenShift crée une route pour l'application conteneurisée.
Scénario Alternatif :	Aucun.
Scénario d'erreur :	Il n'y a pas assez d'espace au niveau des nœuds, le pod ne peut pas être crée.

Tableau 3.6: Description textuelle du cas d'utilisation « Récupérer image »

Acteur :	Développeur
Précondition :	image existante au niveau de Docker registry.
Post condition :	image disponible sur le poste client.
Description :	Le développeur a la possibilité de récupérer des images existante dans le docker registry.
Scénario nominal :	1. Le développeur effectue une commande «docker pull» en précisent l'adresse du docker registry. 2. L'image est récupéré couche par couche sur le poste client
Scénario Alternatif :	Aucun.

Scénario d'erreur :	l'adresse précisé du Docker registry est erroné, l'image ne peut pas être récupérée.

Tableau 3.7: Description textuelle du cas d'utilisation « Récupérer URL de production »

Acteur :	Client
Précondition :	URL exposée, application testé.
Post condition :	Application accessible via URL.
Description :	Le client clique sur l'URL exposé par OpenShift et l'application est accessible dans le navigateur.
Scénario nominal :	1. Le client clique sur l'URL. 2. L'URL s'ouvre dans le navigateur et affiche l'application.
Scénario Alternatif :	Aucun.
Scénario d'erreur :	Pas de routeur déployé sur le nœud sur lequel le pod de l'application est déployé, l'URL n'est pas fonctionnel.

Tableau 3.8: Description textuelle du cas d'utilisation « Récupérer URL de test »

Acteur :	Testeur.
Précondition :	URL exposée.
Post condition :	Application accessible via URL.
Description :	Le testeur clique sur l'URL exposé par OpenShift et l'application est accessible dans le navigateur.
Scénario nominal :	1. Le testeur clique sur l'URL. 2. L'URL s'ouvre dans le navigateur et affiche l'application.

Scénario Alternatif :	Aucun.
Scénario d'erreur :	Pas de routeur déployé sur le nœud sur lequel le pod de l'application est déployé, l'URL n'est pas fonctionnel.

3.3.3 Configuration

Le module de configuration consiste en la création d'un Template pour chaque projet qui contient les configurations nécessaires pour son build et déploiement sur OpenShift et en la création d'un script pipeline qui définit les étapes par lesquelles passe le projet (développement, tests , déploiement, etc.). La figure 3.4 est le diagramme de cas d'utilisation illustrant la configuration. Sa description est récapitulé dans le tableau 3.9 pour la création du template et le tableau 3.10 pour la création du script.

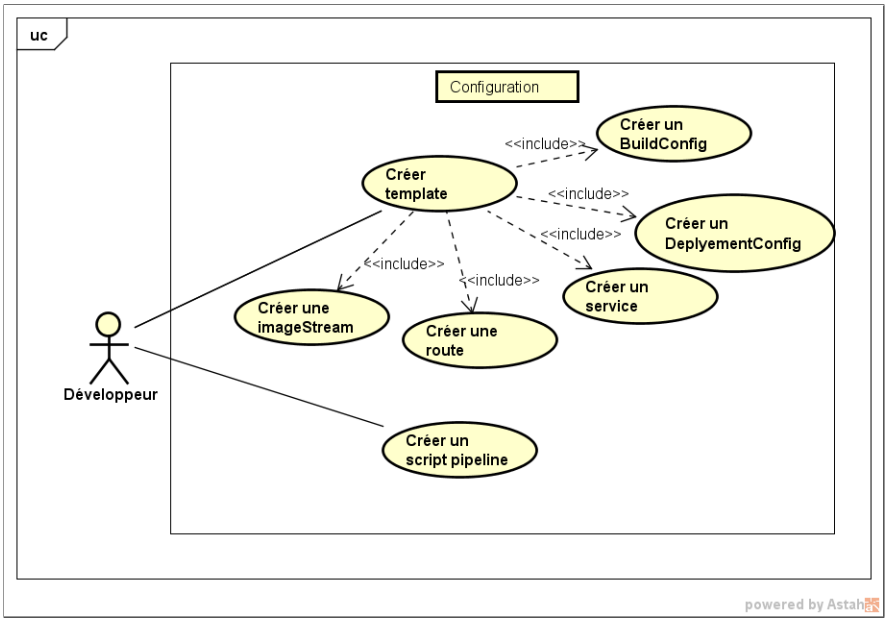


Figure 3.4: Diagramme de cas d'utilisation "Configuration".

Tableau 3.9: Description textuelle du cas d'utilisation « Créer Template »

Acteur :	Développeur.
Précondition :	Connexion au serveur OpenShift.
Post condition :	Les ressources sont créés avec un succès dans OpenShift.

Description :	Le développeur écrit un Template qui décrit le processus de tout ce qui est à générer lors du déploiement d'une application et l'exécute dans un job Jenkins.
Scénario nominal :	1. Le développeur crée un projet de type «free style». 2. Le développeur choisit d'ajouter une ressource OpenShift. 3. Le développeur précise le namespace (nom du projet) au sein duquel le template va être crée. 4. Le développeur saisie le Token d'OpenShift. 5. Le développeur inclut le template. 6. Le développeur lance le job. 7. OpenShift crée les ressources précisé dans le template.
Scénario Alternatif :	Namespace ou token incorrecte, le scénario nominal reprend respectivement aux points 3 ou 4.
Scénario d'erreur :	Le développeur commet une erreur syntaxique, le Template ne peut pas être crée.

Tableau 3.10: Description textuelle du cas d'utilisation « Créer script pipeline »

Acteur :	Développeur.
Précondition :	Connexion établi avec le serveur Jenkins, installation du plugin « OpenShift pipeline plugin » et existence des fichiers BuildConfig et Deployement config au niveau du serveur OpenShift.
Post condition :	Le script est créé avec succès.
Description :	Le script pipeline décrit les étapes que le code va suivre et ce que chaque étape inclut, c'est-à-dire en partant de l'environnement de développement, passant par le test et arrivant jusqu'à la partie déploiement.

Scénario nominal :

1. Le développeur précise sa source maven.
2. Le développeur précise les étapes du pipeline (développement, test et déploiement).
3. Le développeur précise les fichiers à invoquer
4. Le développeur enregistre le script.

Scénario Alternatif : Au niveau de l'étape 2 du scénario nominal, Le développeur précise des étapes du pipeline différent que ceux qui sont fait habituellement. Le scénario nominal reprend au point 3.

Scénario d'erreur :

— Les fichiers à invoquer ne sont pas créé sur OpenShift.Le pipline ne peut pas être crée.

— Les paramètres de connexion aux sources du code (serveur git) ou d'OpenShift sont incorrects.Le pipline ne peut pas être crée.

3.3.4 Supervision

Le module de supervision comprend deux parties, la première est la surveillance de l'état du cluster sur les ressources CPU et RAM, la deuxième est la supervision des états des conteneurs (up, down, suspendu,attente, création de conteneur). La figure 3.5 présente le diagramme de cas d'utilisation du module « monitoring ».

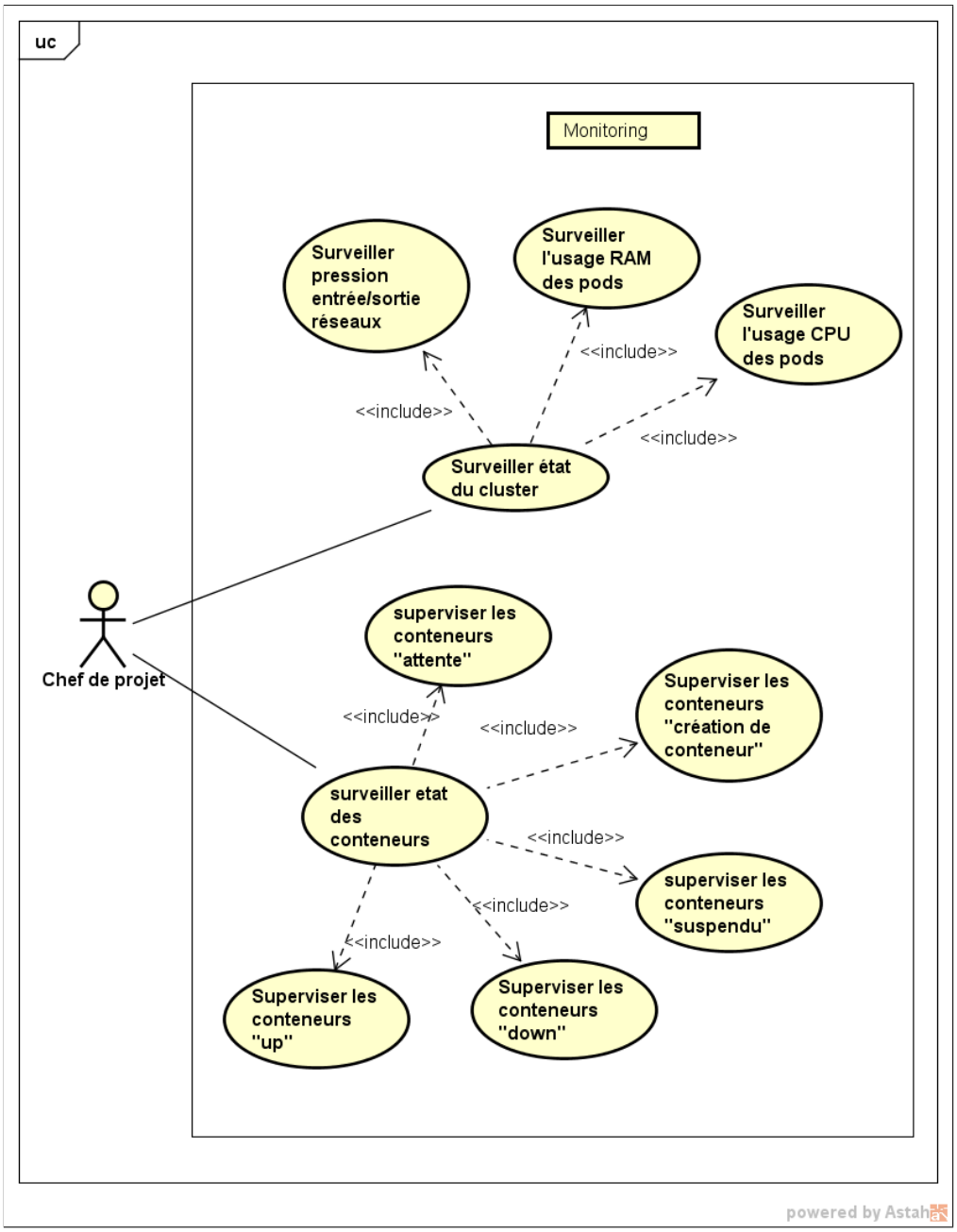


Figure 3.5: Diagramme de cas d'utilisation "Supervision".

Tableau 3.11: Description textuelle du cas d'utilisation «Surveiller état du cluster»

Acteur :	Chef de projet.
Précondition :	connexion au serveur OpenShift.
Post condition :	Affichage d'un tableau de monitoring contenant les éléments décrit.

Description :	Le chef de projet dispose des métriques sur le CPU, la RAM et la pression la pression entrée/sortie sur le réseau à coté de chaque pod.
Scénario nominal :	1. Le chef de projet choisit le projet qu'il désire consulter 2. Le système affiche des chartes graphique indiquant la pression des entrées/sorties sur le réseau, l'usage de la RAM et l'usage du CPU par les pods.
Scénario Alternatif :	Aucun
Scénario d'erreur :	Aucun.

Tableau 3.12: Description textuelle du cas d'utilisation « Surveiller état des conteneurs »

Acteur :	Chef de projet.
Précondition :	Connexion au serveur OpenShift.
Post condition :	Affichage d'un tableau de monitoring affichant les pods ayant l'état « up », « down », « pending » "waiting" et « container creating » .
Description :	le chef de projet peut disposer d'un tableau de bord lui permettant de connaitre quel sont les pods non fonctionnel et ceux qui le sont.
Scénario nominal :	1. Le chef de projet clique sur l'URL exposé par OpenShift pour le tableau de monitoring. 2. Le système affiche un tableau qui regroupe les pods par projet et indique pour chacun son état.
Scénario Alternatif :	Aucun.
Scénario d'erreur :	Aucun.

3.3.5 Pré-configuration

Le volet pré-configuration consiste en la mise en place du pipeline d'intégration continue et du déploiement continu.

La mise en place d'un pipeline d'intégration continue a été faîte suivant deux scénarii, le premier en utilisant pour chaque serveur une machine virtuelle, le deuxième en déployant chaque serveur comme un conteneur Docker sur OpenShift.

La documentation de toute cette partie, a pour finalité de proposer des cas d'utilisation pratiques lors de la pré-configuration de ce pipeline ou de pipeline similaire Le diagramme 3.6 présente les différents cas d'utilisation pour le volet pré-configuration.

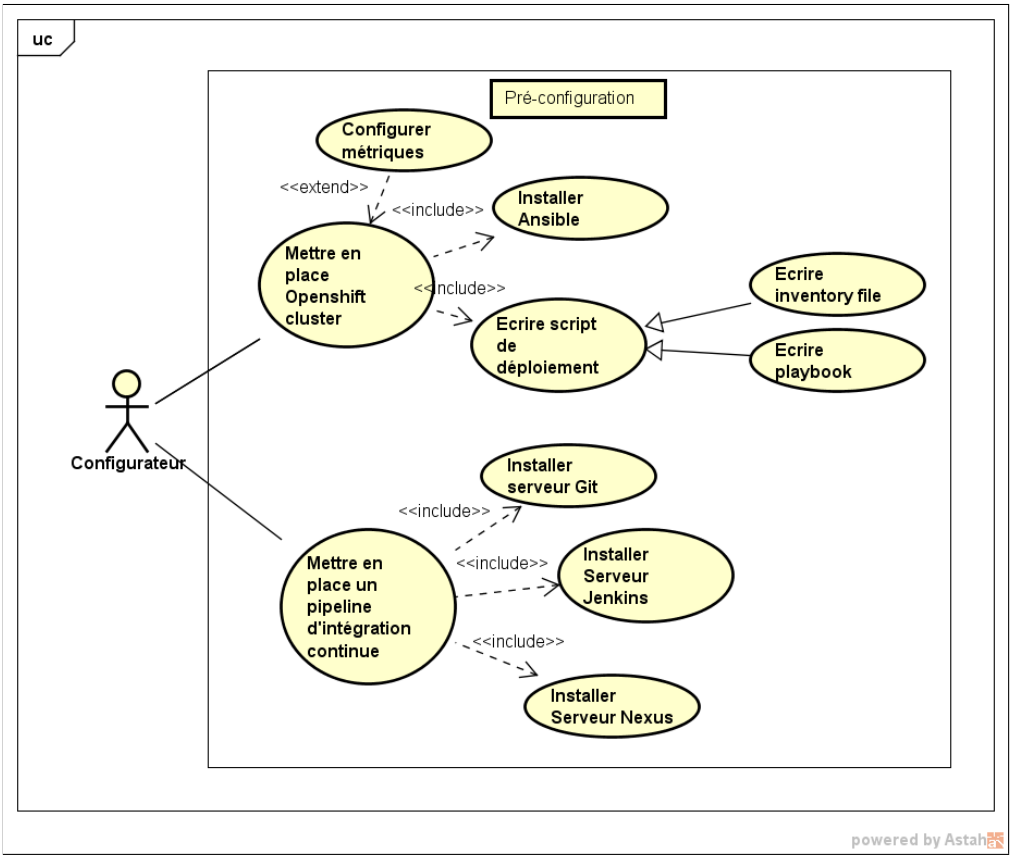


Figure 3.6: Diagramme de cas d'utilisation "Pré-configuration".

Tableau 3.13: Description textuelle du cas d'utilisation "Mettre en place un cluster OpenShift"

Acteur :	Configurateur
Précondition :	Installation de python 2.7 ou 2.6, connexion ssh établit entre le serveur Ansible et les nœuds du futur cluster, connexion internet disponible.

Post condition :	Déploiement d'un cluster OpenShift avec succès.
Description :	La mise en place du cluster OpenShift est l'étape qui permet de mettre en place un pipeline de déploiement continu. Cette étape nécessite la création d'un script Ansible pour l'automatisation de la configuration des nœuds du cluster.
Scénario nominal :	1. Installer l'outil Ansible . 2. Créer un script yaml appelé "playbook" qui contient les opérations à exécuter sur les nœuds cibles . 3. Créer un script yaml appelé "inventory" file qui permet de lister les nœuds qui sont concernés par les configurations existantes dans le "playbook". 4. Exécuter le "playbook".
Scénario Alternatif :	Aucun
Scénario d'erreur :	Playbook ou inventory file mal configuré, le cluster ne peut pas être déployé.

Avant de décrire le cas d'utilisation « configurer métrique » nous expliquons quelques termes et leurs utilités.

Pour pouvoir afficher les métriques sur le CPU, La RAM et la pression entrée/sortie du réseau , il faut déployer les conteneurs hawkular pour le stockage des métriques, Cassandra pour leurs persistances, Heapster pour leurs extractions et Hawkular OpenShift Agent pour pouvoir faire tout ceci sur tous les nœuds du cluster.

Le service account permet la consommation de l'API d'OpenShift sans Token. Ceci est utile pour permettre à un conteneur d'accéder à un autre.

Tableau 3.14: Description textuelle du cas d'utilisation "Configurer métriques"

Acteur :	Configurateur
Précondition :	Cluster OpenShift déployé.
Post condition :	Affichage des métriques aux cotés des pods

Description : La configuration des métriques permet l'affichage des métriques sur la RAM, le CPU et la pression entrée/sortie du réseau ainsi que le statut des pods.

Scénario nominal :

1. Le configurateur crée un service Account.
2. Le configurateur procure des droits de modifications au service Account crée.
3. Le configurateur donne à Heapster le droit d'accès aux nœuds du cluster.
4. Le configurateur crée un volume persistant et précise sa taille.
5. Le configurateur déploie un template Cassandra.
6. OpenShift crée un conteneur Cassandra.
7. Le configurateur génère des certifications.
8. Le configurateur déploie les composantes des métriques.
9. OpenShift crée un conteneur Hawkular et un conteneur Heapster.
10. Le configurateur précise l'URL d'accès aux métriques au niveau des fichiers de configuration d'OpenShift master.
11. Le configurateur déploie Hawkular Openshift Agent.
12. Openshift crée un conteneur Hawkular OpenShift Agent.
13. Le configurateur déploie Kubernetes Dashboard.
14. OpenShift crée un conteneur Kubernetes Dashboard.

Scénario Alternatif : Aucun

Scénario d'erreur :	l'un des composants , cassandre, hawkular, hawkular Openshift Agent, Heapster n'est pas déployé, les métriques ne peuvent pas être affichées.

Tableau 3.15: Description textuelle du cas d'utilisation "Installer serveur Git"

Acteur :	configurateur
Précondition :	connexion internet disponible.
Post condition :	serveur Git fonctionnel.
Description :	La mise en place du serveur Git est la première étape pour le déploiement d'une chaîne d'intégration continue.
Scénario nominal 1 :	1. Installer la commande git. 2. Créer un dossier .ssh. 3. Attribuer des droits lecture/écriture/exécution à l'utilisateur de la session. 4. Générer la clé publique. 5. Copier la clé publique de tous les clients Git dans le fichier authorized_keys du serveur Git. 6. Créer un dépôt Git.
Scénario nominal 2 :	1. Récupérer template gitServer persistant. 2. Donner aux conteneurs du cluster le droit d'accès au conteneur Git. 3. Configurer les paramètres de connexion au serveur Git. 4. écuriser la route de Git. 5. Désactiver le clonage automatique à partir de Git.

Scénario	Utiliser le protocole http au lieu de ssh.
Alternatif1 :	
Scénario	Utiliser template non persistent, le scénario nominal 1
Alternatif2 :	reprend au point 2.
Scénario d'erreur 1 :	Connexion non disponible, installation de la commande git impossible.
Scénario d'erreur 2 :	Aucun.

Tableau 3.16: Description textuelle du cas d'utilisation "Installer serveur Jenkins"

Acteur :	Configurateur
Précondition :	Connexion internet disponible.
Post condition :	Serveur Jenkins accessible via le navigateur.
Description :	La mise en place du serveur Jenkins non pas en tant que service mais sous un servlet, dans notre cas apache tomcat.
Scénario nominal 1 :	1. Installer Java. 2. Configurer les variables d'environnement JAVA_HOME, JRE_HOME et PATH. 3. Installer apache tomcat. 4. Rendre l'exécution de tomcat en tant que service. 5. Rendre le script exécutable. 6. Déployer jenkins.war sur apache tomcat
Scénario nominal 2 :	1. Récupérer template jenkins persistant. 2. Désactiver SeLinux.
Scénario Alternatif 1 :	Aucun.

Scénario Alternatif 2 :	Récupérer template Jenkins non persistant, le scénario nominal 2 reprend au point 2.
Scénario d'erreur 1 :	Fichier de configuration pour l'exécution de tomcat en tant que service mal configuré, apache tomcat est impossible à démarrer.
Scénario d'erreur 2 :	Aucun.

Tableau 3.17: Description textuelle du cas d'utilisation "Installer serveur Nexus"

Acteur :	Configurateur
Précondition :	Aucune.
Post condition :	Serveur Nexus accessible via le navigateur.
Description :	Mise en place du serveur Nexus qui sera notre dépôt Maven.
Scénario nominal 1 :	1. Installer Openjdk8. 2. Modifier l'utilisateur de Nexus dans le fichier de configuration. 3. Installer apache tomcat. 4. Rendre l'exécution de nexus en tant que service. 5. Redémarrer la machine.
Scénario nominal 2 :	1. Récupérer template Nexus. 2. Modifier volume pour la persistance.
Scénario Alternatif 1 :	Aucun.
Scénario Alternatif 2 :	Aucun.
Scénario d'erreur 1 :	Aucun.
Scénario d'erreur 2 :	Aucun.

Tableau 3.18: Description textuelle du cas d'utilisation « Mettre en place une chaîne d'intégration continue »

Acteur :	Configurateur
Précondition :	Les serveurs Git, Nexus et Jenkins sont installés, maven installé sur le serveur Jenkins.
Post condition :	Les artefacts sont déployés sur Nexus, et un build est déclenché sur Jenkins.
Description :	La chaîne d'intégration continue est fonctionnelle, si un push effectué par le développeur sur le serveur Git déclenche un build sur le serveur Jenkins, et déploie les artefacts générés suite à un build réussi sur le serveur Nexus.
Scénario nominal :	1. Etablir une connexion ssh entre les serveurs Git, Jenkins et Nexus. 2. Configurer le fichier "pom.xml" du projet pour pointer sur le répertoire du serveur Nexus. 3. Configurer le fichier "settings.xml" de maven en indiquant l'utilisateur, le mot de passe et l'identifiant du serveur Nexus.
Scénario Alternatif :	Aucun.
Scénario d'erreur :	Fichier de configuration "pom.xml" et "settings.xml" ne contiennent pas le même identifiant Nexus. Les artéfacts ne peuvent pas être déployés sur Nexus.

Conclusion

Cette partie nous a permis dans un premier lieu d'identifier les acteurs de notre système, ensuite nous avons spécifié les besoins fonctionnels et non fonctionnels de notre système avant de clôturer avec l'analyse des besoins.

Chapitre 4

Architecture et conception détaillée

Plan

1 Architecture générale de la solution 52

2 Aspect dynamique de la solution 57

Introduction

Nous nous intéressons dans ce chapitre à l'architecture générale de notre système, nous passons, par la suite, à la description de la vue dynamique de notre projet que nous détaillons pour chaque volet.

4.1 Architecture générale de la solution

Pour répondre au besoin préalablement exprimé, nous avons proposé et mis en place deux architectures physiques différentes nécessaires pour répondre aux exigences spécifiques de l'entreprise hôte.

4.1.1 Architecture générale basée sur les machines virtuelles

La première architecture que nous avons proposé est basée sur les machines virtuelles. En effet, chaque serveur est hébergé dans une machine virtuelle qui lui est consacré. Cette approche permet de disposer d'un pipeline fonctionnel indépendant de toute plateforme. La figure 4.1 est un schéma explicatif du pipeline intégration et déploiement continus.

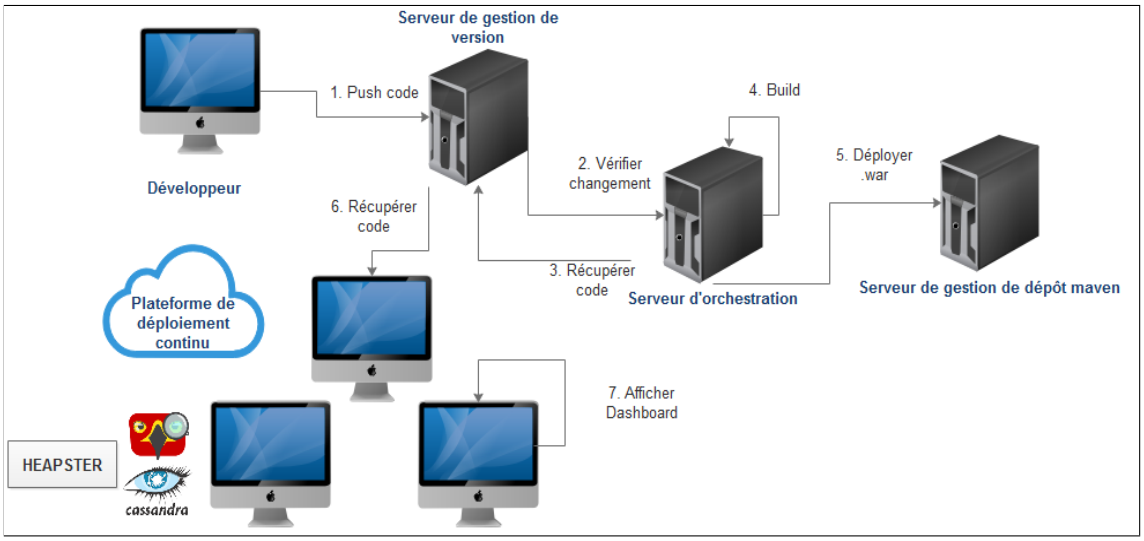


Figure 4.1: Schéma explicatif du pipeline intégration et déploiement continus.

Heapster, Hawkular et cassandra sont des conteneurs déployés sur OpenShift . Heapster est responsables de l'extraction des métriques à partir du cluster OpenShift. Hawkular est un engin de stockage de métriques qui procure des moyens de création, d'accès et de gestion historiées des métriques. Enfin, cassandra stocke les données qui proviennent d' Hawkular et permet leur

persistance.

Le diagramme de déploiement représenté par la figure 4.2 décrit la disposition physique de chaque entité préalablement identifiée dans le schéma explicatif de notre pipeline, il décrit aussi, les protocoles de connexions entre ces différentes composantes.

Le schéma 4.2 contient aussi un certain nombre d'éléments essentiels au bon fonctionnement du pipeline, nous définissons :

- OpenVswitch : c'est un switch virtuel qui permet aux applications qui tournent sur un hyperviseur (machine virtuelle) d'avoir à disposition les fonctionnalités d'un switch physique mais sur des liens réseaux virtuels, entre la machine virtuelle et l'hyperviseur.[15]
- Router : c'est un routeur qui permet le routage des paquets entre les différents nœuds du cluster.
- Deployer : ce conteneur permet à une application d'être déployé, c'est l'entité qui déploie le pod au sein duquel l'application désiré sera déployé.
- Kubelet : c'est un composant appartenant à Kubernetes, il permet de s'assurer que tous les conteneurs au sein du nœud sont fonctionnels.
- Scheduler : c'est un composant qui permet de répartir les pods sur les différents nœuds du cluster.
- Kubernetes API server : c'est un composant qui permet de fournir une API REST structurée et interrogeable.
- Replication controller : c'est le composant responsable de recréer les conteneurs lorsque ce dernier échoue.
- OpenShift client tool : c'est un composant qui permet la consommation de l'API via le terminal. OpenShift client tool offre plus de possibilités pour manipulation du cluster que la console.

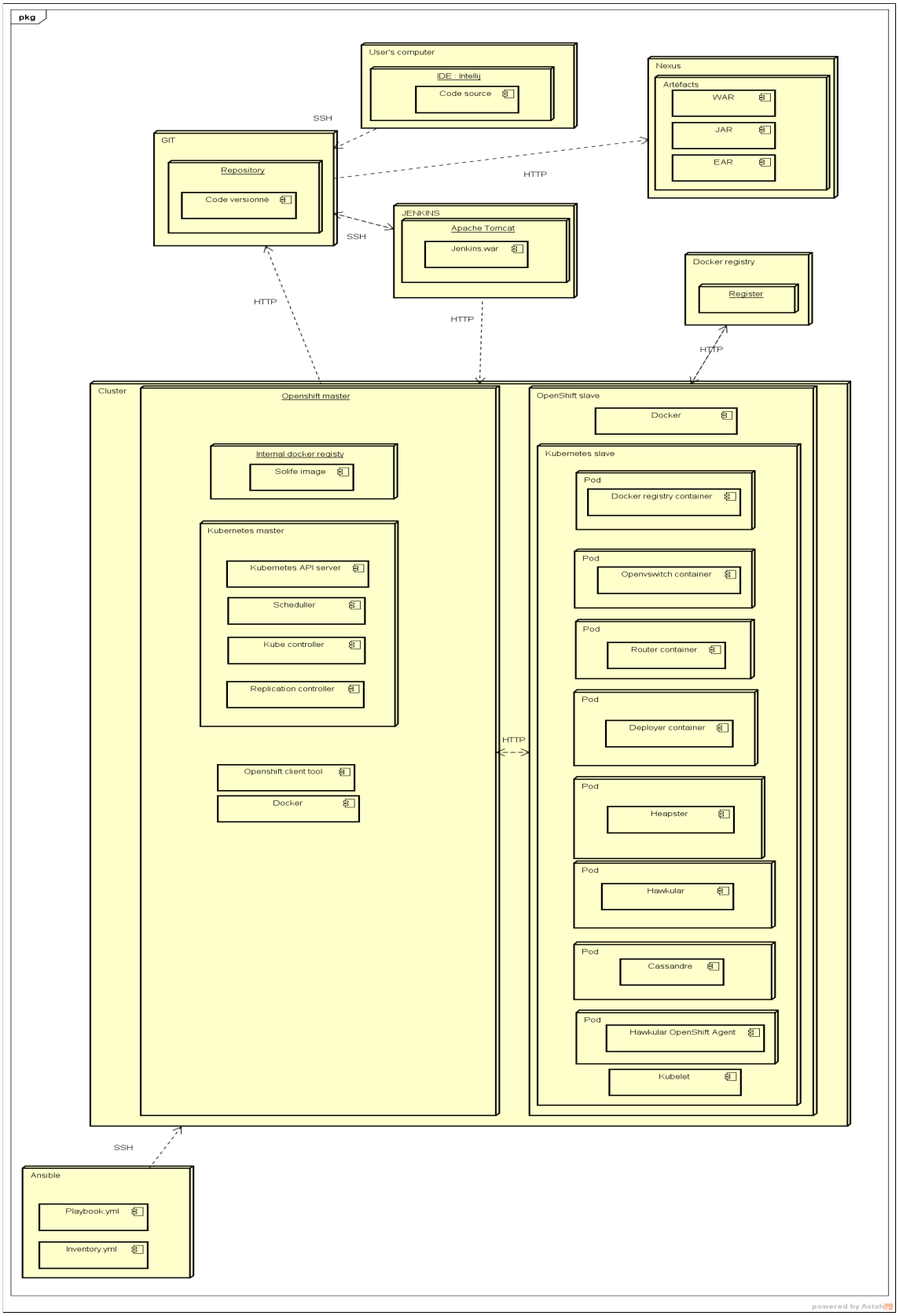


Figure 4.2: Diagramme de déploiement pour l'architecture à base de machines virtuelles.

4.1.2 Architecture générale basée sur les conteneurs docker

La deuxième architecture que nous avons proposé est celle où, tout est conteneurisé dans des conteneurs docker monté sur OpenShift. Cette architecture permet de profiter de l'avantage que propose OpenShift à savoir le gain en espace, sans avoir une machine qui héberge le serveur git ou Jenkins. Il suffit, donc d'un conteneur docker qui prend moins d'espace mémoire et aussi moins de temps à démarrer. Avec à cette architecture les serveurs Nexus, Jenkins et Git font désormais partie des conteneurs existant sur OpenShift et peuvent donc être supervisés.
La figure 4.3 présente le schéma explicatif du pipeline d'intégration et déploiement continus à base de conteneurs dockers montés sur OpenShift.

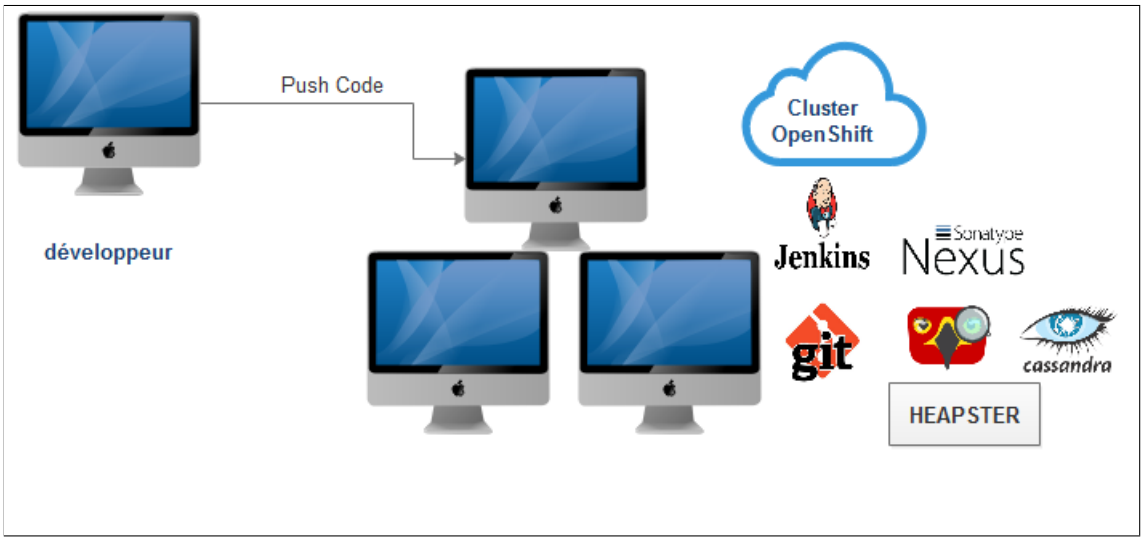


Figure 4.3: Schéma explicatif du pipeline intégration et déploiement continus à base de conteurs Docker.

Le diagramme de déploiement présenté par la figure 4.4, met en évidence l'architecture physique et le protocole de communication entre les différentes entités. Nous remarquons que tous les serveurs, à savoir, Git, Jenkins, Nexus, sont conteneurisés en plus de Hawkular, Cassandra et Heapster. Il est à savoir qu'au sein d'OpenShift cluster, les différents pods communiquent entre eux grâce au protocole http.

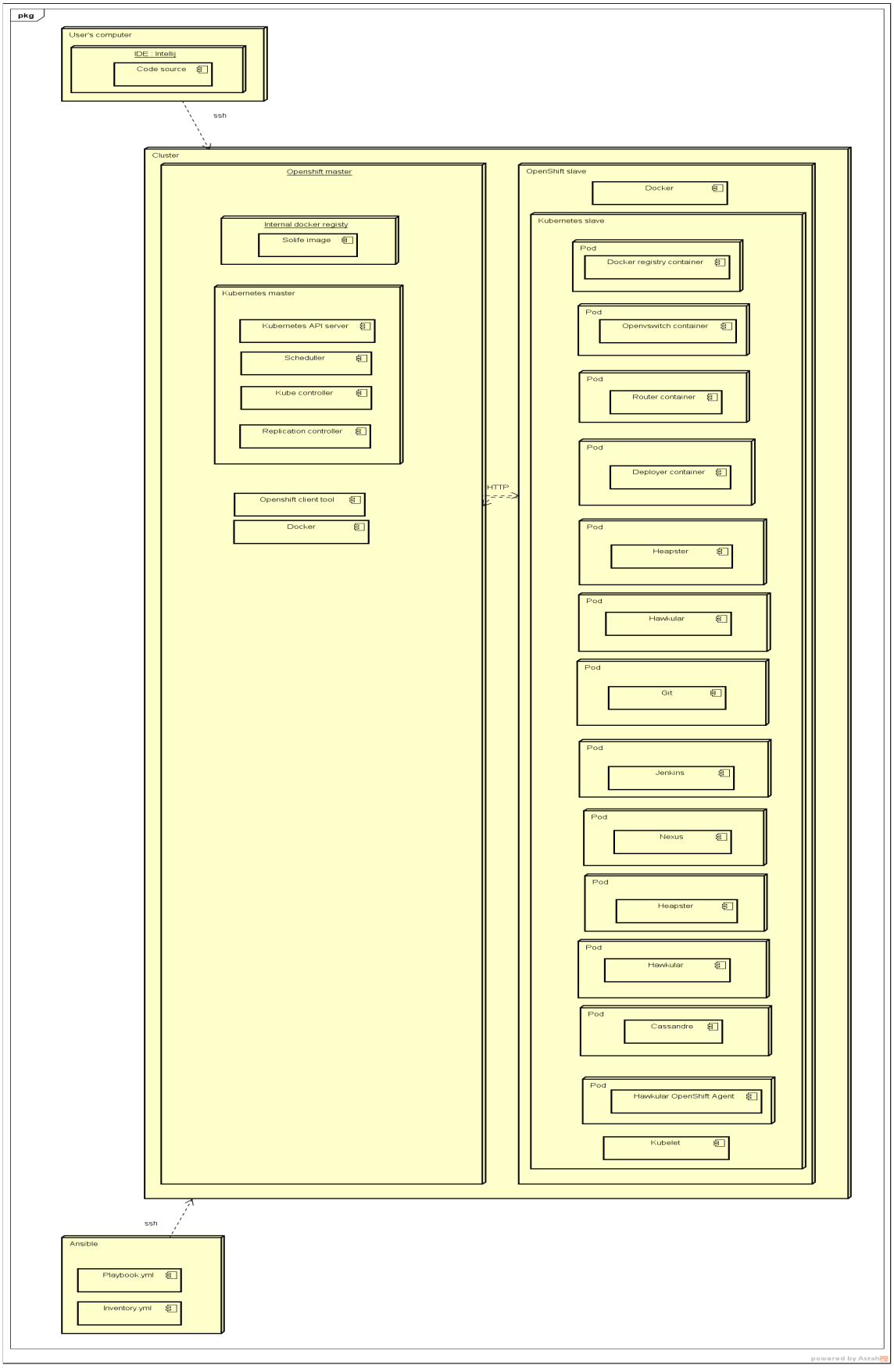


Figure 4.4: Diagramme de déploiement pour l'architecture à base de conteneurs Docker

4.2 Aspect dynamique de la solution

Pour mieux comprendre l'interaction entre les différents composants de l'architecture, nous présentons une vue dynamique de notre système.

4.2.1 Vue dynamique du volet "Intégration continue"

Le diagramme d'activité illustré dans la figure 4.5, explique le fonctionnement et l'interaction entre les différents composants du pipeline intégration continue.

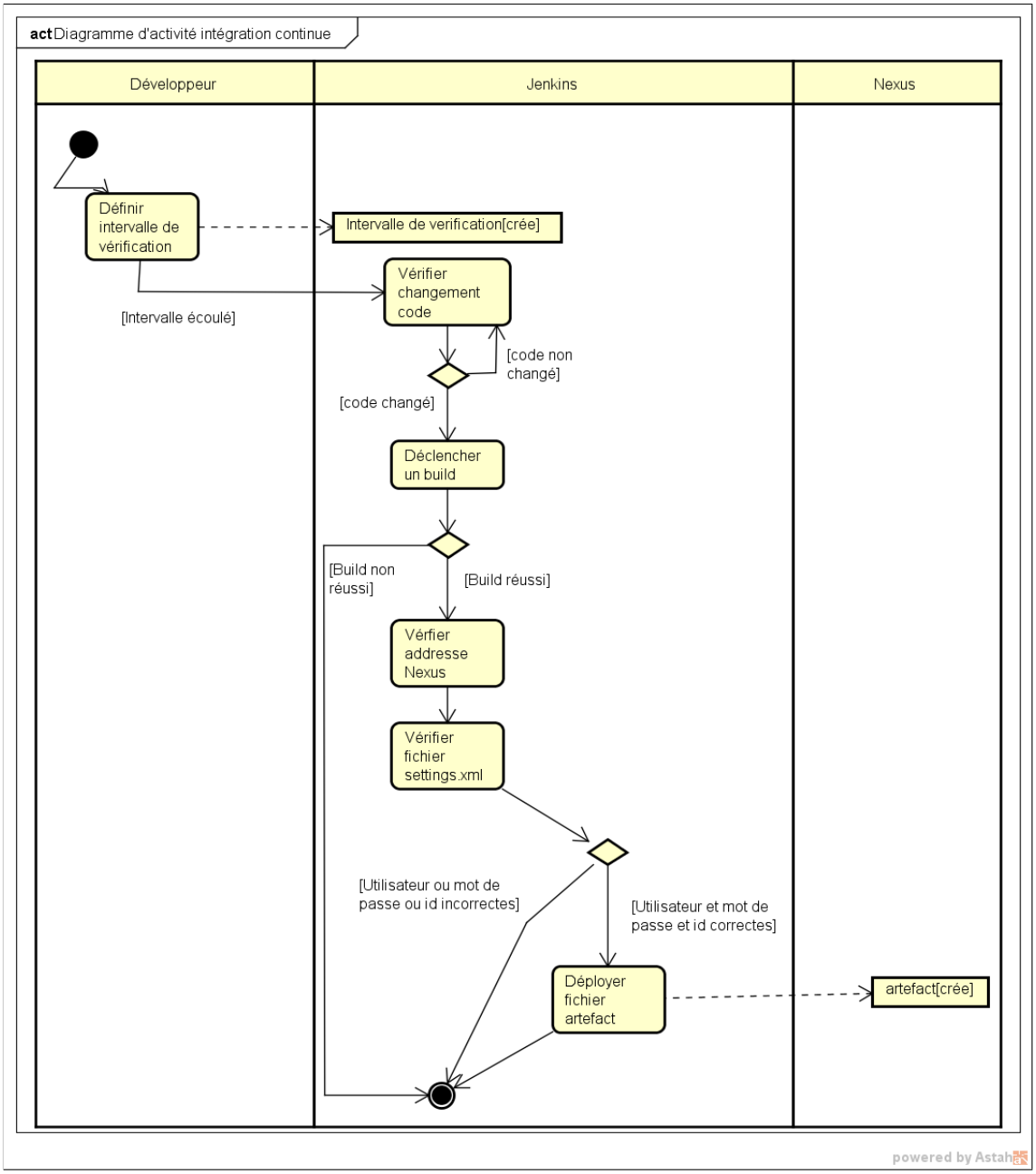


Figure 4.5: Processus d'Intégration continue

4.2.2 Vue dynamique du volet "Déploiement continu"

Le diagramme d'activité représenté dans la figure 4.6, illustre la vue dynamique du cas d'utilisation « déployer application ». En précondition, le build précédemment déclenché doit être effectué avec succès, ainsi, le nœud master d'OpenShift choisit le nœud slave au sein duquel le pod sera créé. Ce choix, se base sur les ressources nécessaires pour la création du pod et les ressources disponibles dans les nœuds du cluster. Une fois le nœud choisi, une adresse IP interne est alors assigné au Pod, cette adresse sera reconnue uniquement à travers le cluster OpenShift. Ensuite, OpenShift instancie l'image docker qu'il a préalablement récupéré du docker registry suite au build réussi, ainsi un conteneur est créé.
OpenShift expose le service du conteneur et lui crée une route afin qu'il soit accessible pour les utilisateurs.

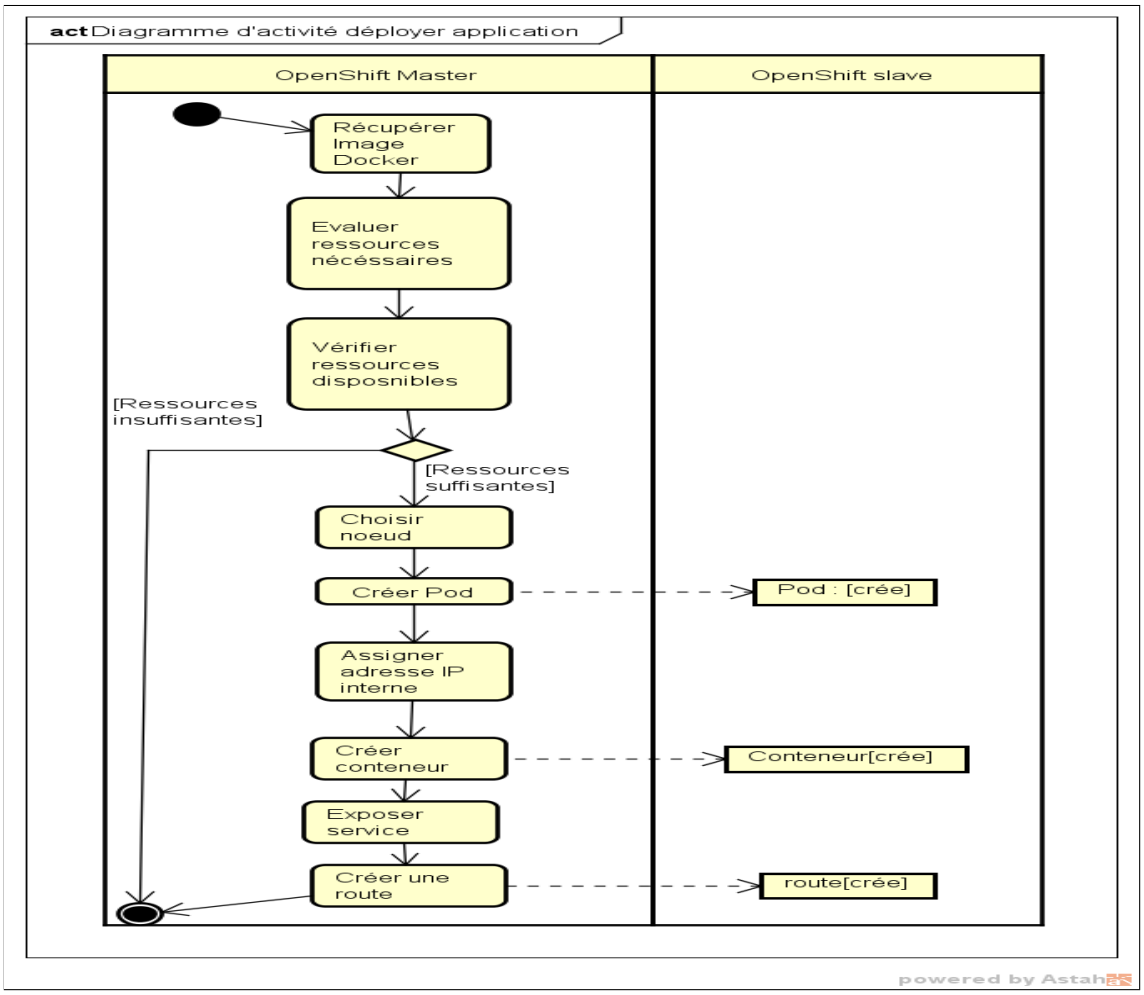


Figure 4.6: Processus de déploiement d'une application

4.2.3 Vue dynamique du volet "Configuration"

le diagramme d'activité représenté par la figure 4.7, illustre le cas d'utilisation exposé dans le chapitre précedent.

— BuildConfig : c'est un fichier ou partie d'un template qui permet de préciser la source à partie de laquelle le code sera récupéré.

— DeployementConfig : c'est un fichier ou partie d'un template qui permet de préciser la politique de déploiement d'un conteneur.

— Namespace : c'est le nom du projet au sein d'OpenShift.

— Service : c'est le service qu'OpenShift expose pour permettre à l'application d'être visible à l'extérieur du cluster OpenShift.

— Route : c'est l'adresse URL à partir de laquelle l'application est accéssible.

— ImageStream : c'est l'image généré par OpenShift suite à un build réussi.

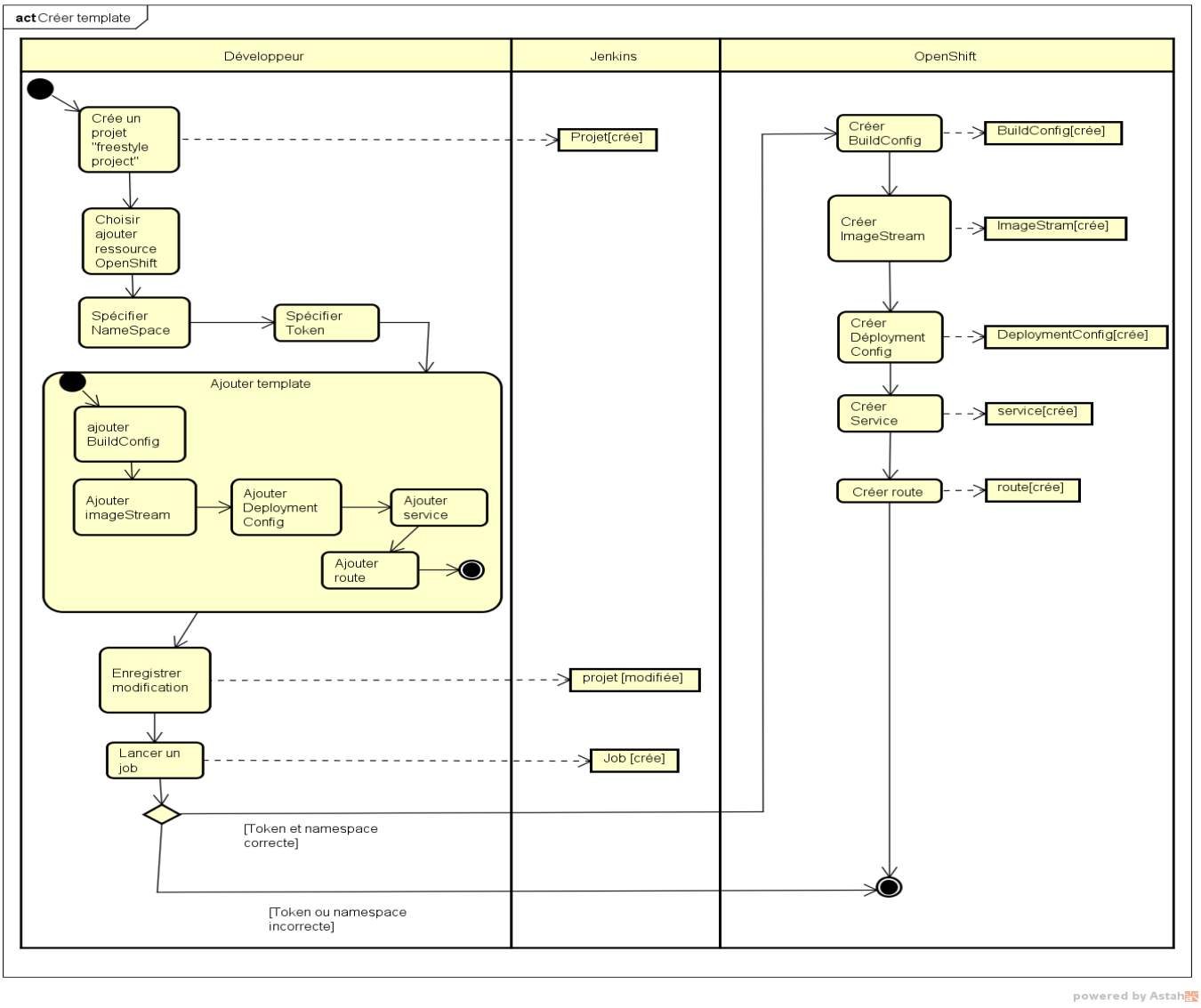


Figure 4.7: Processus de création d'un template

4.2.4 Vu dynamique du volet "Pré-configuration"

Pour mieux comprendre le module de pré-configuration, nous proposons les diagrammes d'activité présentés par les figures 4.8 , 4.9 et 4.10 , qui expliquent respectivement les étapes à faire pour l'installation du serveur Git, pour l'installation du serveur Jenkins et pour l'installation du serveur Nexus, ceci avec la méthode à base de machines virtuelle et la méthode à base de conteneur. Le volet pré-configuration inclut aussi des configurations pour l'extraction des métriques, ce processus est représenté par la figure 4.11.

La présentation de ce volet a deux finalités, la première est d'expliquer le comportement qui a été mis en œuvre pour le contexte de ce pipeline. La deuxième finalité, étant donnée les nombreuses

contraintes techniques rencontrés et les difficultés surmontés, ceci pourrait constituer un guide pour ceux qui sont confronté à des contraintes techniques similaires.

4.2.4.1 Installation Serveur Git

Pour installer le serveur Git, nous avons procédé de deux manières, une à base de machine virtuelle et une à base de conteneur.

Installation Serveur Git à base de machines virtuelles : Le processus de l'installation du serveur Git contient les étapes suivantes, établissement de la connexion ssh entre la machine Git et ses clients et la création d'un dépôt Git.

Installation Serveur Git à base de conteneurs : Le processus de déploiement d'un conteneur Git sur OpenShift est le déploiement d'un template Git ainsi que la sécurisation de l'accès à ce conteneur avec le Protocole TLS.

La figure 4.8 est le diagramme d'activité pour l'installation du serveur Git.

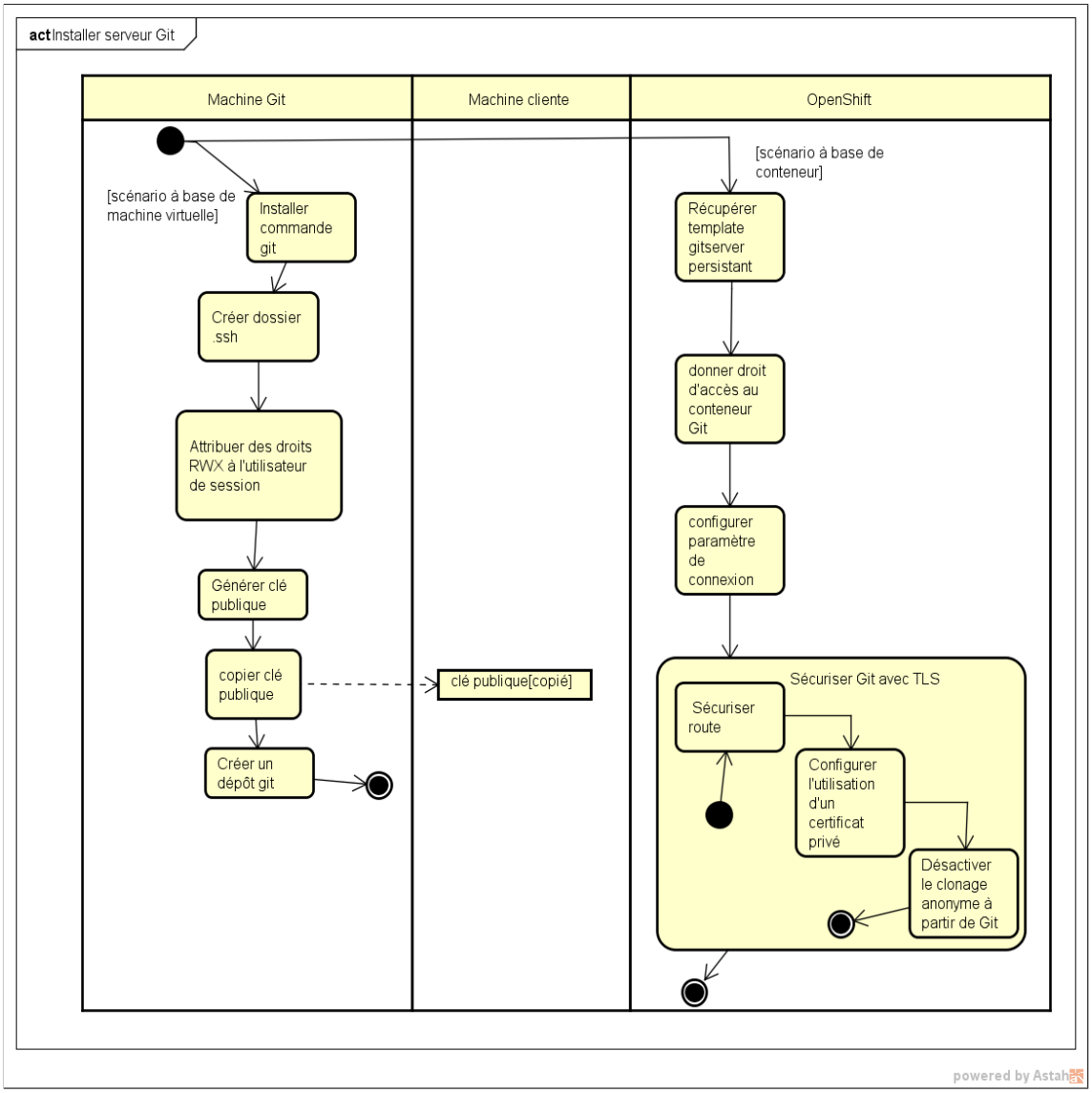


Figure 4.8: Processus d'installation du serveur Git

4.2.4.2 Installation Serveur Jenkins

Pour installer le serveur Jenkins, nous avons procédé de deux manières, une à base de machine virtuelle et une à base de conteneur.

Installation Serveur Jenkins à base de machines virtuelles : Le procédé d'installation du serveur Jenkins se fait en déployant un fichier jenkins.war sur un serveur apache tomcat.

Installation Serveur Jenkins à base de conteneur : Avec cette architecture, le processus est la récupération d'un template jenkins persistant et la désactivation du SeLinux dans la machine OpenShift Master.

La figure 4.9 est le diagramme d'activité pour l'installation du serveur Jenkins.

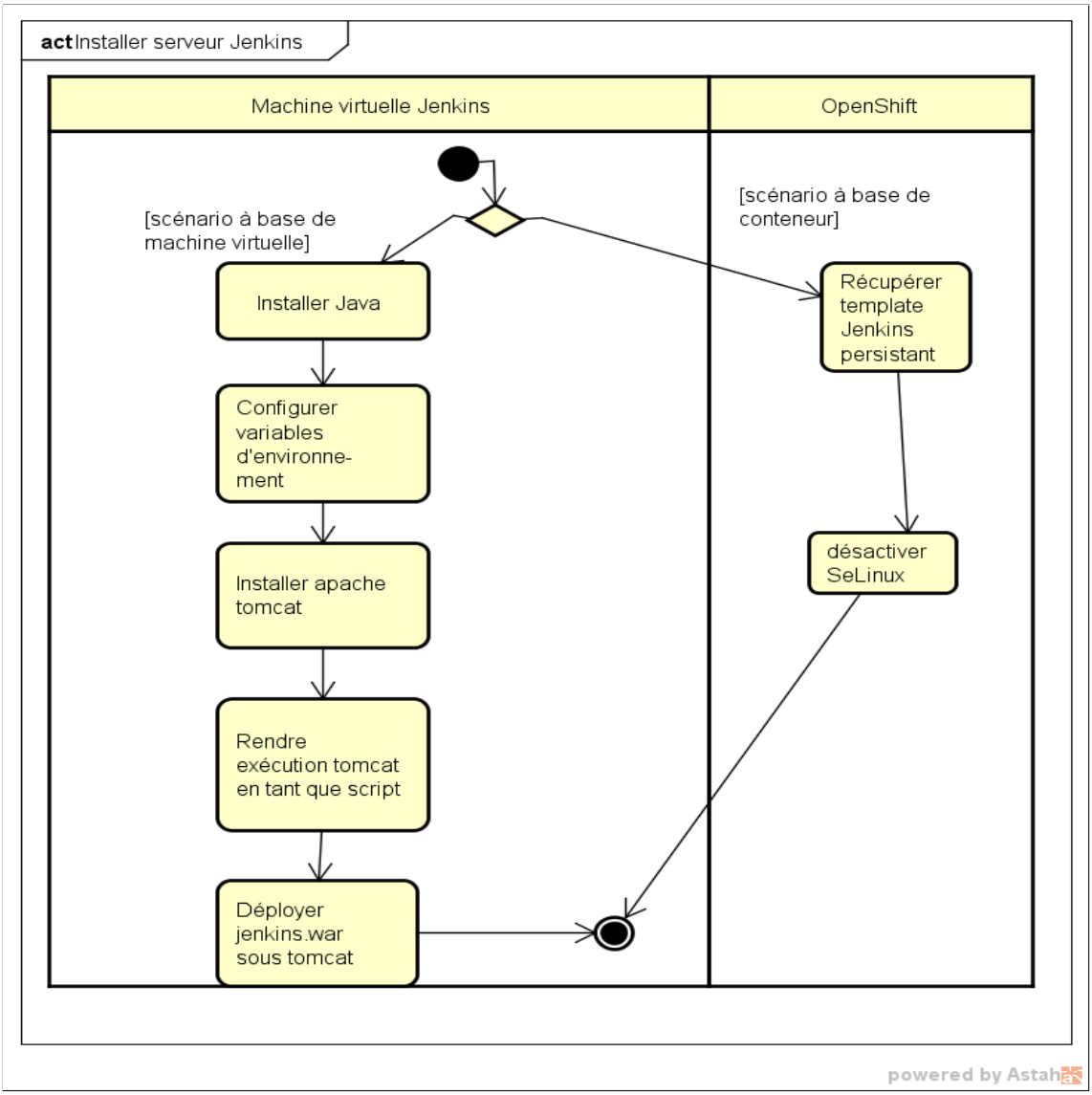


Figure 4.9: Processus d'installation du serveur Jenkins

4.2.4.3 Installation Serveur Nexus

Pour installer le serveur Nexus, nous avons procédé de deux manières, une à base de machine virtuelle et une à base de conteneur.

Installation Serveur Nexus à base de machines virtuelles : Le processus d'installation de Nexus sur une machine virtuelle, comme par l'installation d'OpenJDK, ensuite l'attribution d'un utilisateur non « sudoer » au serveur et efin la modification l'exécution de nexus, en une exécution en tant que service.

Installation Serveur Nexus à base de conteneur : Le processus de déploiement de Nexus comme étant un conteneur sur OpenShift, nécessite la récupération d'un template Nexus puis configurer sa persistance.

La figure 4.10 est le diagramme d'activité pour l'installation du serveur Nexus.

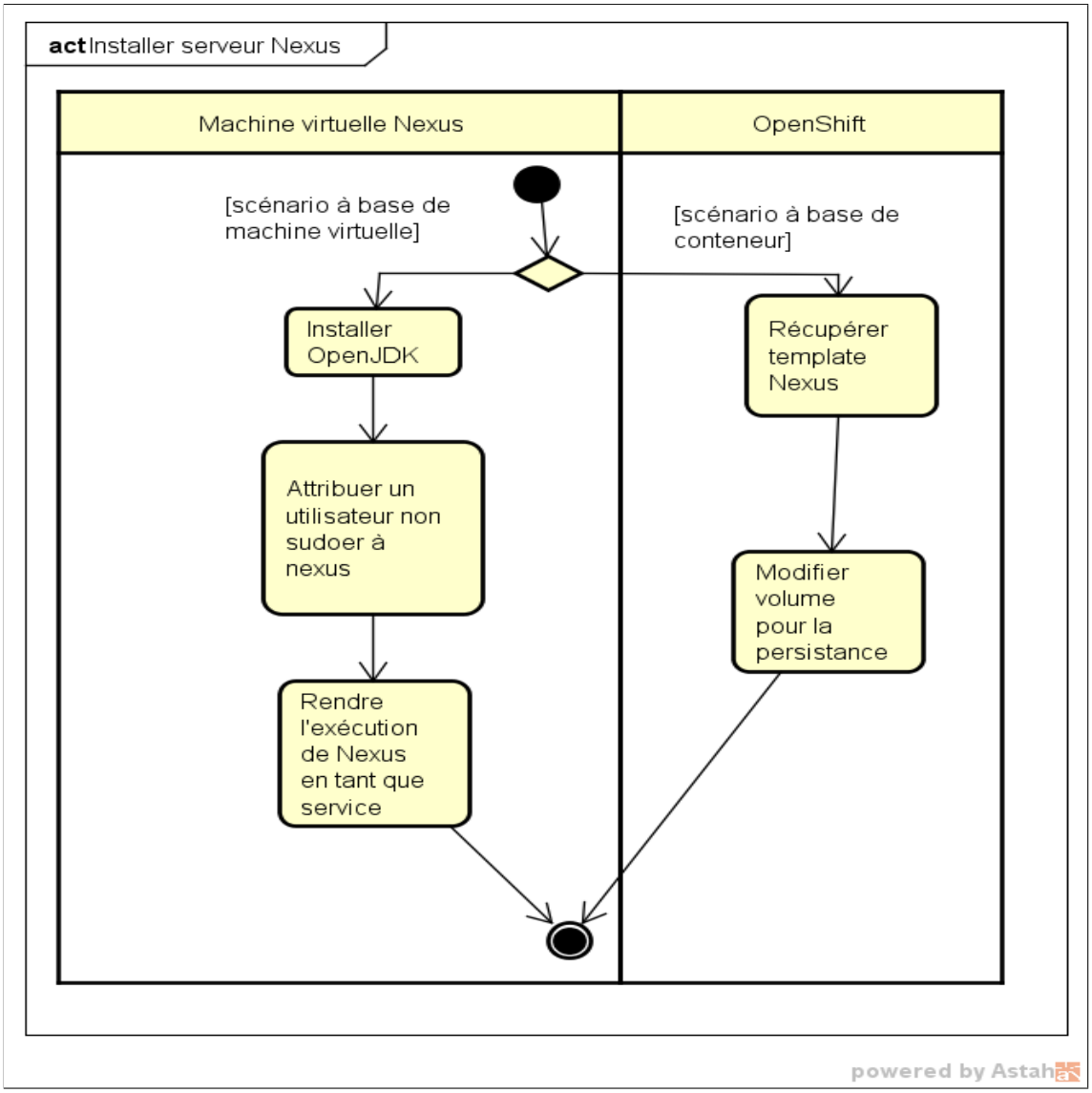


Figure 4.10: Processus d'installation du serveur Nexus

4.2.4.4 Configuration des métriques

La figure 4.11, présente le processus nécessaire pour l'extraction et l'affichage des métriques sur OpenShift et sur le dashboard de Kubernetes.

Le cluster role est une permission, qui donne à son détenteur le pouvoir absolue dans le cluster. Quant au deamonSet, il est utilisé pour déployer des réplicas d'un pod sur un ou plusieurs nœuds

du cluster, comme déployer un agent de monitoring sur les différents nœuds du cluster.

Pour pouvoir extraire et afficher les métriques sur OpenShift, il faut, tout d'abord, créer un service Account de déploiement qui sera utilisé pour pouvoir déployer toutes les composantes des métriques. Ensuite, pour que Heapster puisse lister tous les nœuds du cluster et consommer les métriques qui sont exposé par chaque nœud, il faut lui donner le rôle de « cluster reader ».
Il faut maintenant persister les données extraites, une base de donnée «Cassandra» est nécessaire mais sans l'affecter à un volume persistant et la précision de sa taille, à chaque fois que le cluster redémarrera nous perdrons toutes les données stockés.
Le service Account préalablement crée, nécessite des certifications sur le cluster, si ces dernières ne sont disponible, il suffit de générer des certifications.
Ensuite, il faut déployer les composantes des métriques, ce qui créera les conteneurs Hawkular et Heapster, avant de déployer Hawkular OpenShift Agent, il faut au niveau du nœud master d'OpenShift, préciser l'URL d'accès aux métriques.

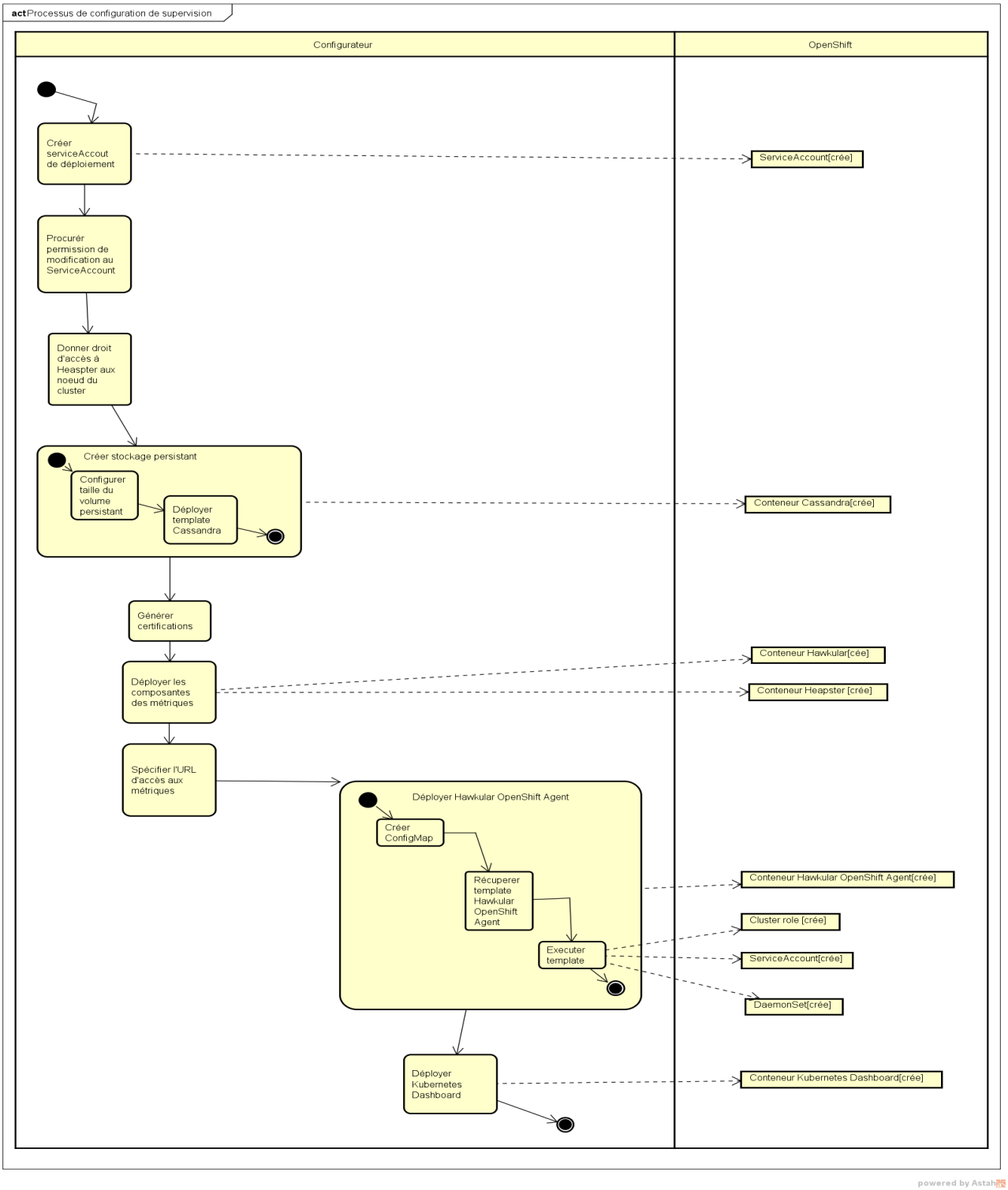


Figure 4.11: Processus de supervision de l'état du cluster

4.2.4.5 Processus de la pré-configuration

Pour mieux comprendre la pré-configuration, nous proposons un diagramme d'activité présenté par la figure 4.12, qui explique les étapes à faire au niveau de chaque serveur pour aboutir au résultat

escompté.

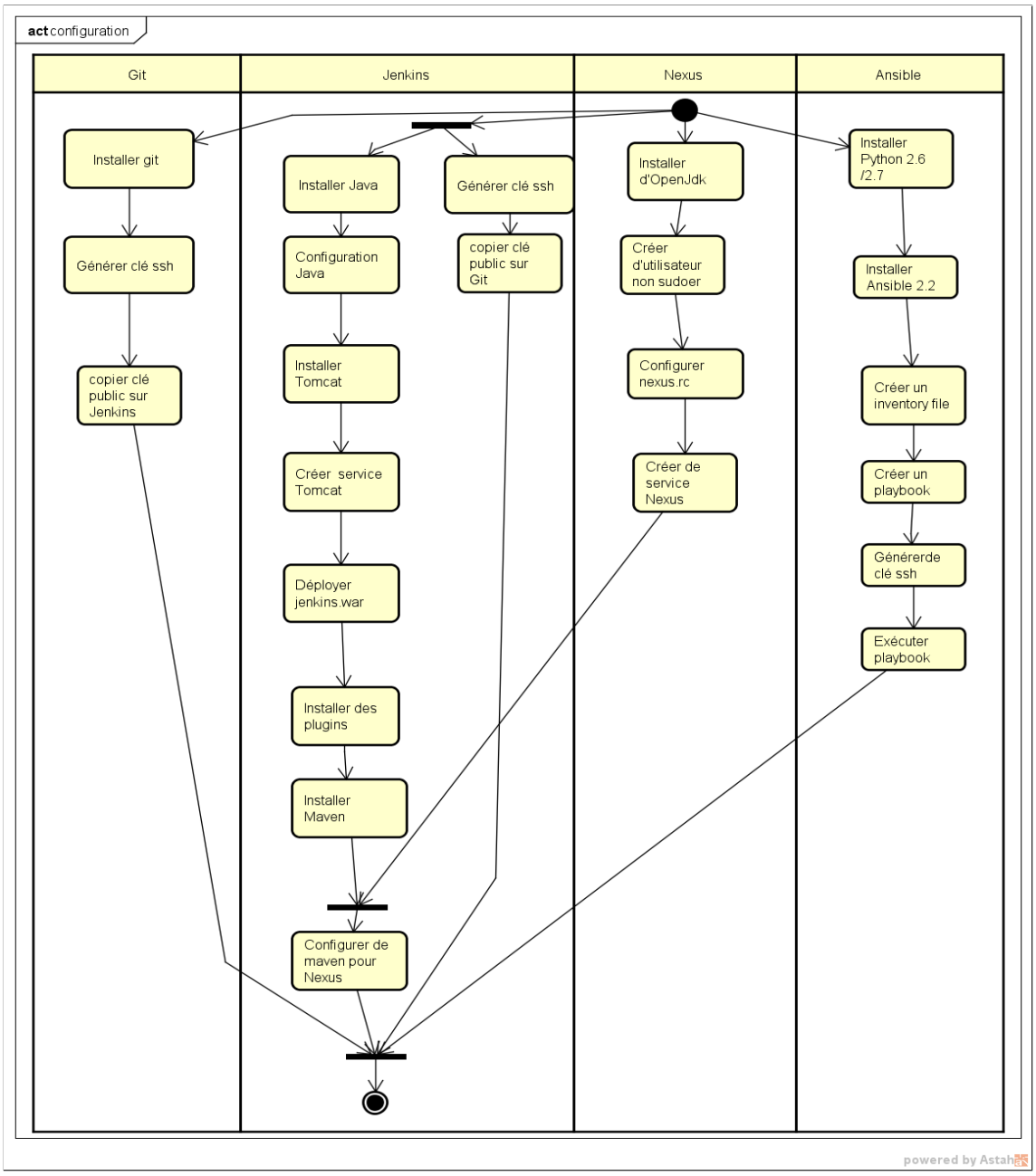


Figure 4.12: Processus pour la pré-configuration.

La documentation de toute cette partie de pré-configuration, a pour finalité de proposer des cas d'utilisation de bonnes pratiques lors de la pré-configuration de ce pipeline ou de pipeline similaire.

4.2.4.6 Processus général d'intégration et de déploiement continus dans le pipeline

Le diagramme d'activité nous permet de mieux comprendre l'enchainement des évènements dès le moment où le développeur expose son code sur Git jusqu'au moment où l'application est accessible par le client. La figure 4.13 présente le diagramme d'activité.

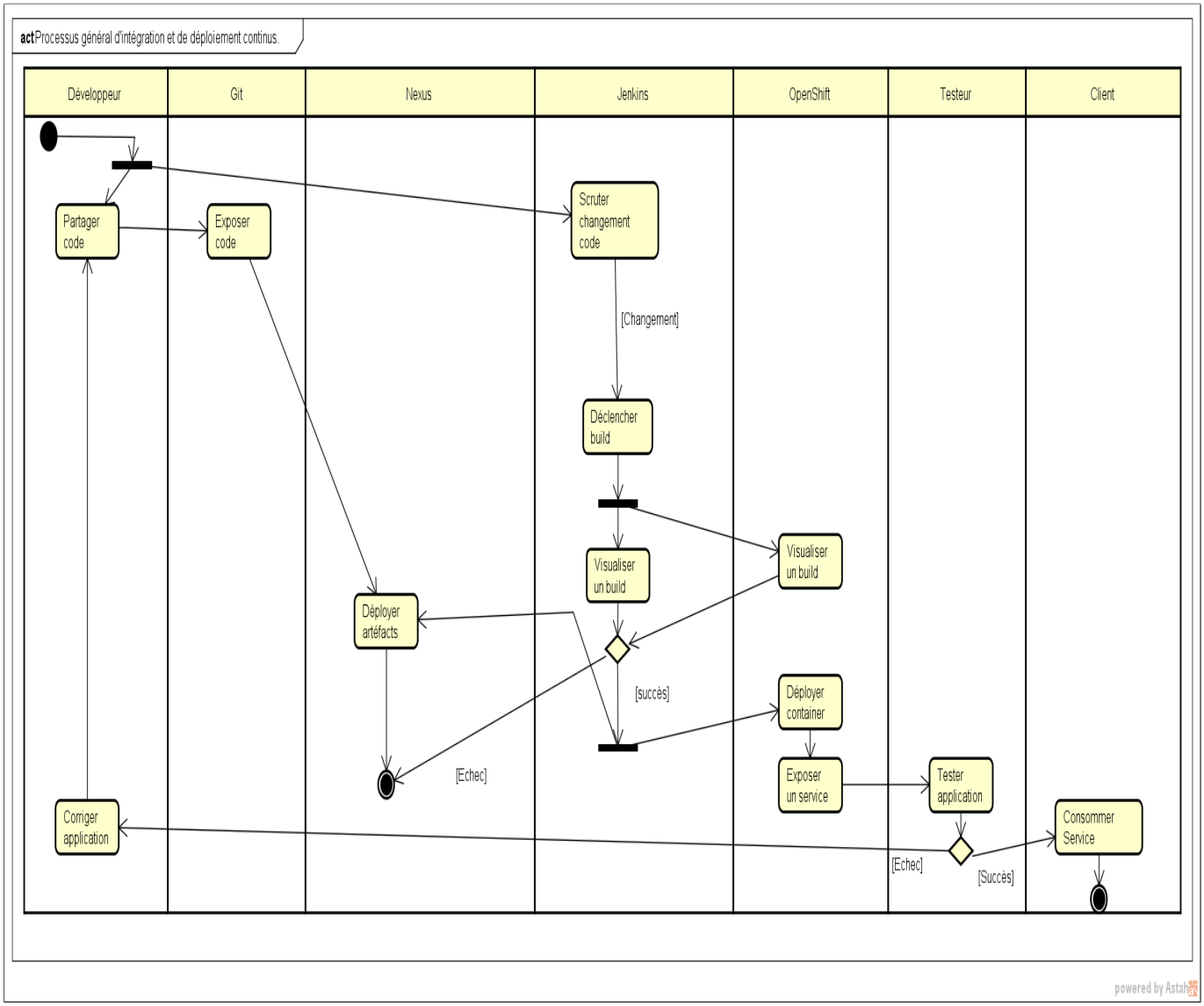


Figure 4.13: Processus d'intégration et de déploiement continus dans le pipeline

Ce diagramme met en évidence les différentes actions qui sont effectués soit, par le développeur et /ou le chef de projet pour déclencher le pipeline, soit par le système lui-même.
Le pipeline est déclenché par un « push » effectué par le développeur sur le système de gestion de version, dans notre cas Git.

Le changement au niveau de ce dépôt déclenche un build au niveau de Jenkins, si le build est réussi, un fichier .war est déposé au niveau du serveur Nexus.
Le code subit une batterie de tests pour empêcher tout problème de régression, une fois ces tests sont validés, un conteneur est créé au niveau d'OpenShift et une URL est exposée permettant ainsi au client d'accéder à l'application.

Conclusion

Dans le présent chapitre nous avons expliqué les différentes architectures générales de notre solution, ensuite nous avons présenté l'aspect dynamique de cette dernière. Le chapitre suivant portera sur la réalisation de notre travail

Chapitre 5

Réalisation

Plan

1 Environnement de travail 71

2 Technologies utilisées 72

3 Principaux développements 74

Introduction

Après avoir traité la partie conceptuelle, nous entamons la partie réalisation de notre projet où nous présentons l'environnement logiciel et matériel de notre travail, ensuite nous exposons les différentes technologies utilisées pour la mise en place de notre système. Enfin, nous présentons les principaux développements de notre projet à travers l'essentiel des imprimes écran de nos interfaces.

5.1 Environnement de travail

Pour mener à bien notre projet, nous avons utilisé un ordinateur FUJITSU dont les caractéristiques techniques sont les suivantes :

— Mémoire : 12GB RAM

— Système d'exploitation : Windows 7 Professional.

— Processeur :Intel(R) Core(TM) i3-3240, 3,40GHz

— Disque dur : 340GB

Nous avons aussi utilisé "VMware workstation 12" pour les machines virtuelles et nous avons mis en place 6 machines. Le tableau 5.1 présente les caractéristiques techniques de chacune d'entre elles.

Tableau 5.1: Caractéristiques techniques des machines virtuelles

Machine	**Mémoire**	**Système d'exploitation**	**Processeur**	**Disque dur**
Git	1GB RAM	CentOS 6	1GHz	20GB
Jenkins	1GB RAM	CentOS 6	1GHz	20GB
Nexus	1GB RAM	CentOS 6	1GHz	20GB
Ansible	512MB RAM	CentOS 6	1 GHz	20GB
OpenShift master	2GB RAM	atomic CentOS 7	1 GHz	40GB
OpenShift slave	4GB RAM	atomic CentOS 7	1 GHz	40GB

5.2 Technologies utilisées

Dans cette section nous exposons la liste des technologies utilisés pour la réalisation de notre travail.

5.2.1 Technologie de configuration : YAML

Pour le développement des scripts, nous avons utilisés le langage de sérialisation YAML qui fournit de puissants paramètres de configuration, sans avoir à apprendre un type de code plus complexe comme CSS, JavaScript et PHP.
YAML est l'acronyme de «Yet Another Markup Language», il se définit comme étant « un standard de sérialisation de données pour tous les langages, facile à utiliser pour les humains ».[16]

— Standard : car il n'y a pas d'approximation à avoir sur le contenu écrit en YAML.

— Sérialisation de données : consiste en la transformation d'une structure de données en un fichier texte.

— Human-friendly : une structure de données écrite en YAML est très simple à lire, facile à comprendre, et facile à éditer.

5.2.2 Technologie de virtualisation : Docker et machine virtuelle

Comme nous l'avons déjà énoncé au cours du chapitre "Étude préalable", nous avons opté pour la combinaison entre les deux techniques de virtualisation, Docker et machines virtuelle.

5.2.2.1 Docker

Docker permet de créer des environnements appelées conteneurs qui permettent d'isoler des applications et leurs dépendances, le tout sous forme de conteneur Linux [17]. La version de docker que nous avons utilisé est 1.10.
Nous avons eu recourt à la virtualisation à base de docker pour déployer des applications sur le cluster OpenShift.

5.2.2.2 Machine virtuelle

Une machine virtuelle est un environnement d'application ou de système d'exploitation installé sur un logiciel qui imite un matériel dédié. Pour l'utilisateur final, l'interaction avec une machine virtuelle est identique à celle avec un matériel dédié.[18]

Nous avons utilisé les machines virtuelle pour les serveurs Git, Jenkins, Nexus et Ansible pour le scénario de déploiement de pipeline d'intégration continu à base de machines virtuelles, nous l'avons aussi utilisé pour les nœuds du cluster OpenShift.

5.2.3 Technologie de configuration d'ordinateurs : Ansible

Pour la configuration des ordinateurs à distance, et plus précisément pour le déploiement du cluster OpenShift, nous avons eu recourt à Ansible qui est un outil open-source de gestion de configuration écrit en python. Grâce à un inventory file listant les nœuds cibles et à un playbook énonçant les tâches à effectuer sur ces nœuds, nous avons pu mettre en place notre cluster OpenShift. Nous avons utilisé la version d'Ansible 2.3.

5.2.4 Outil de gestion de version : Git

Pour la gestion de version de code, nous avons utilisé le serveur Git, qui a l'avantage d'être décentralisé et permet de gérer la notion de « Single point of failure » précédemment expliqué dans le chapitre "Étude préalable".

5.2.5 Serveur : Apache Tomcat

Apache Tomcat est un conteneur web libre de servlets JSP JEE. Pour notre projet, nous avons utilisé apache tomcat comme servlet pour déployer notre serveur d'orchestration Jenkins. Nous avons utilisé la version 7 d'apache tomcat.

5.2.6 Outil d'orchestration : Jenkins

Jenkins, initialement connu sous le nom de Hudson, est un outil d'intégration continue open source, écrit en JAVA. Pour notre pipeline, nous avons utilisé la version 2.35 de Jenkins.

5.2.7 Outil de gestion dépôt maven : Nexus sonatype

Nexus est un outil de gestion des dépôts Maven, il permet de stocker les artefacts générés suite à un build réussi, ainsi ces artefacts constituent à la fois un backup et une bibliothèque utilisable par les autres développeurs.
Pour notre pipeline nous avons utilisé Nexus 3.

5.2.8 Plateforme de déploiement : OpenShift

OpenShift est une solution PaaS (Plateforme-as-a-Service) privée qui permet de construire, déployer et exécuter des applications dans des conteneurs. Elle est disponible en open source et est distribuée dans deux produits : Origin (communautaire) ou Enterprise. [19]
Pour notre projet, nous avons utilisé la version communautaire OpenShift 1.5.

5.3 Principaux développements

Dans cette section, nous présentons grâce à un enchaînement de captures d'écrans, des scénarii d'exécution donnant un aperçu sur la réalisation des fonctionnalités de notre système.

5.3.1 Intégration et déploiement continus

Dans cette partie, nous avons intégré le pipeline d'intégration continu avec celui du déploiement continu. Ainsi les imprimes écran qui suivent, illustrent l'enchaînement du processus d'intégration et déploiement continus. La figure 5.1 comporte un imprime écran sur un job jenkins contenant un script pipeline définissant la préparation, le build et le déploiement comme étant les étapes d'un pipeline appliqué sur l'application.

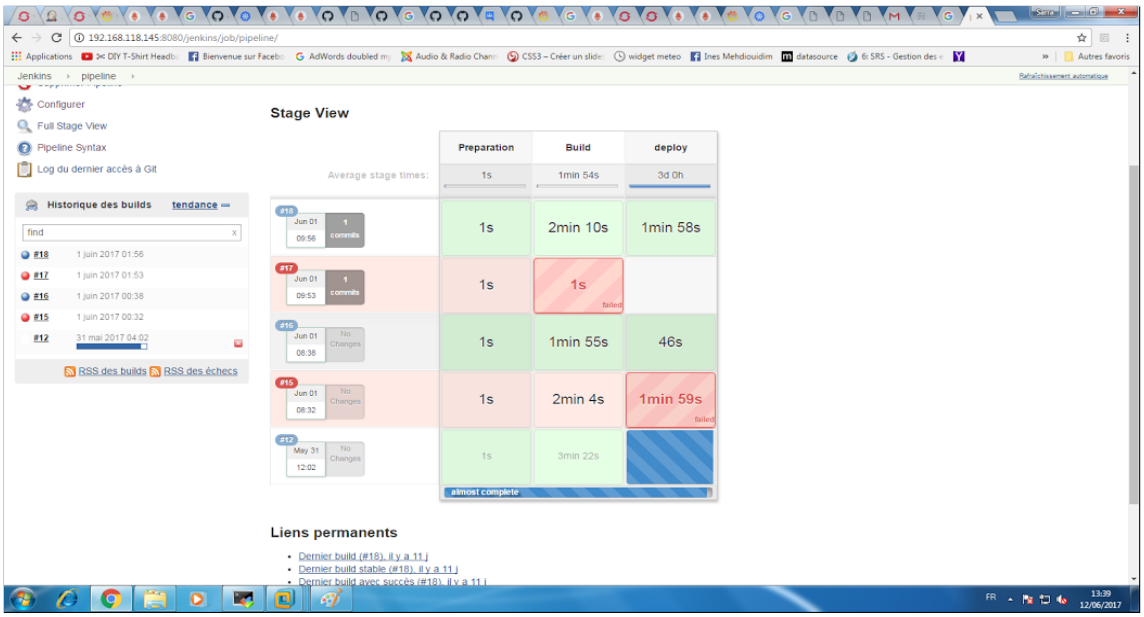


Figure 5.1: Job jenkins de type pipeline.

La figure 5.2 montre le build déclenché par Jenkins et qui s'est déroulé sur OpenShift

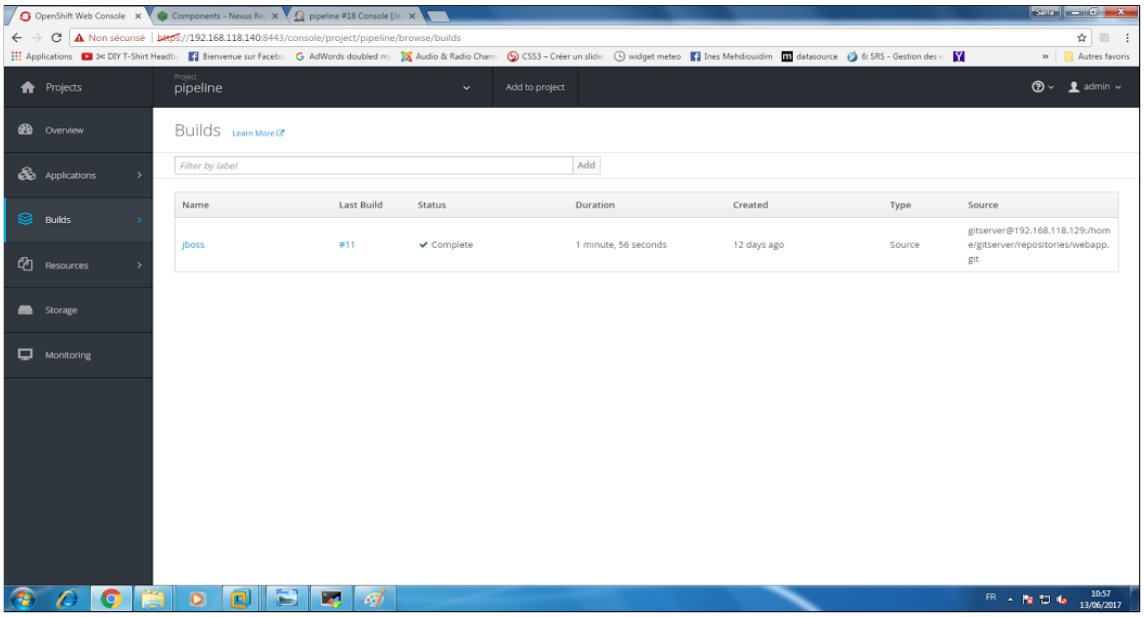


Figure 5.2: Build effectué sur OpenShift.

La figure 5.3 montre les artefacts déployés sur Nexus, suite au build réussi de l'application.

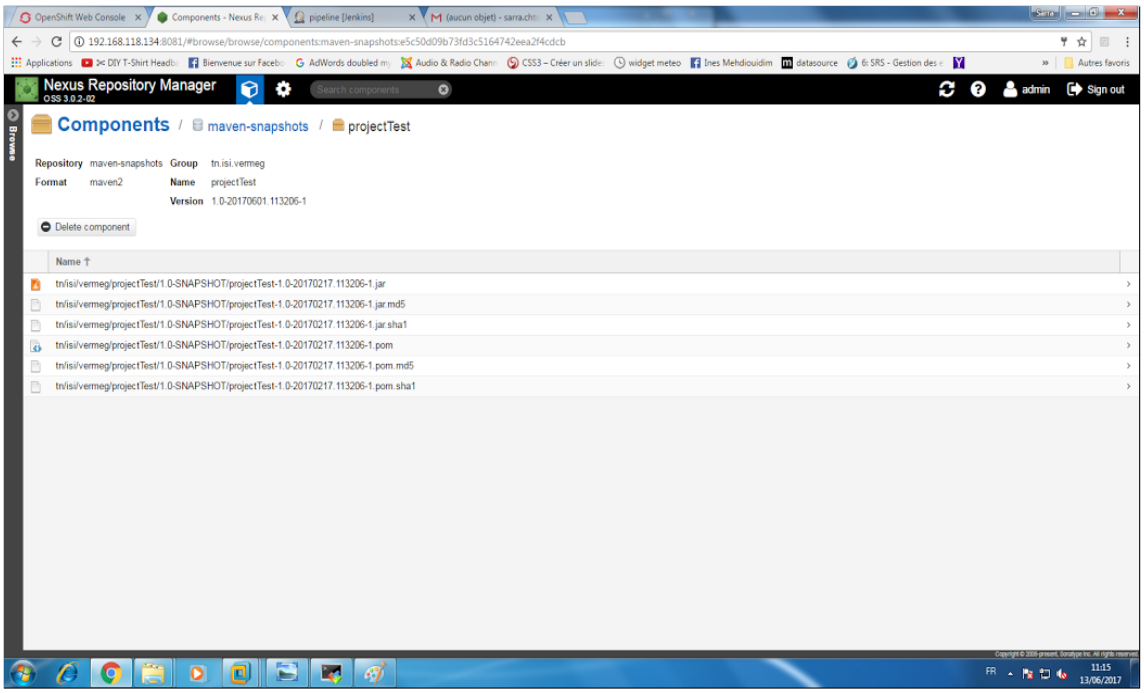


Figure 5.3: Artefacts deployé sur Nexus.

La figure 5.4 montre le déploiement de l'application déclenché par Jenkins et qui s'est déroulé sur OpenShift

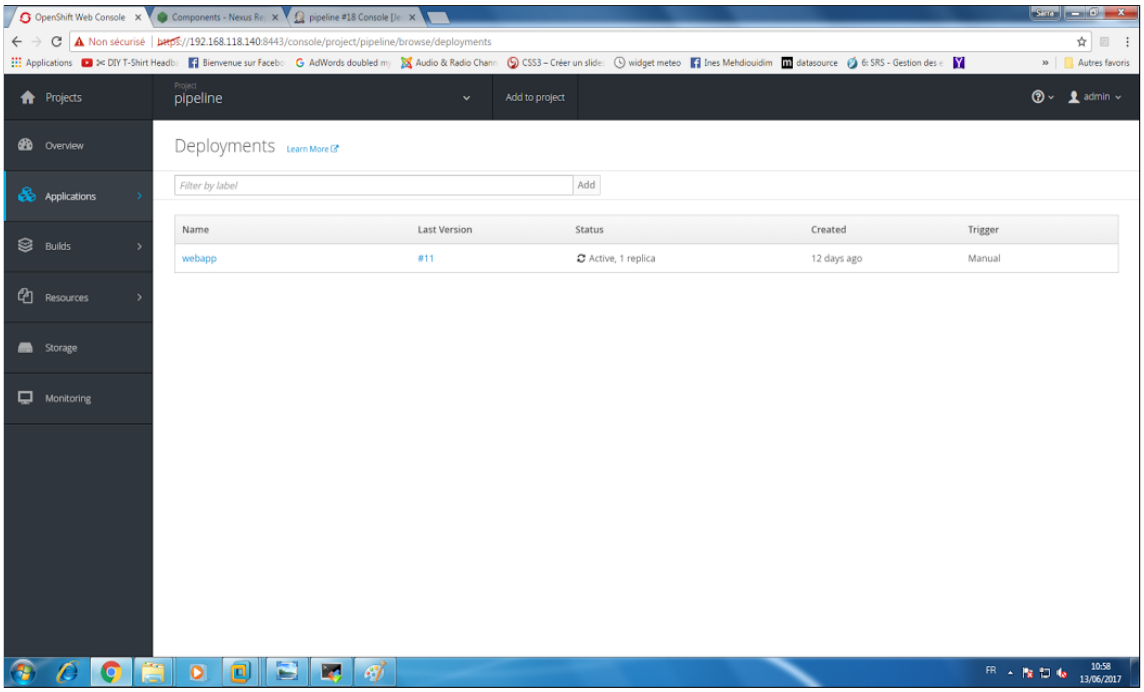


Figure 5.4: Déploiement d'une application sur OpenShift.

5.3.2 Configuration

Le module de configuration est nécessaires au bon déploiement d'une application sur OpenShift. Ce template est créé comme étant un job sur Jenkins, et instancié sur OpenShift. Le module inclut aussi un script pipeline qui décrit les différentes étapes par lesquelles passe le code avant d'être déployé.

L'imprime écran 5.5 explique la création d'un template avec Jenkins et la figure 5.6 son instanciation au sein d'OpenShift une fois que le build du job est effectué avec succès.

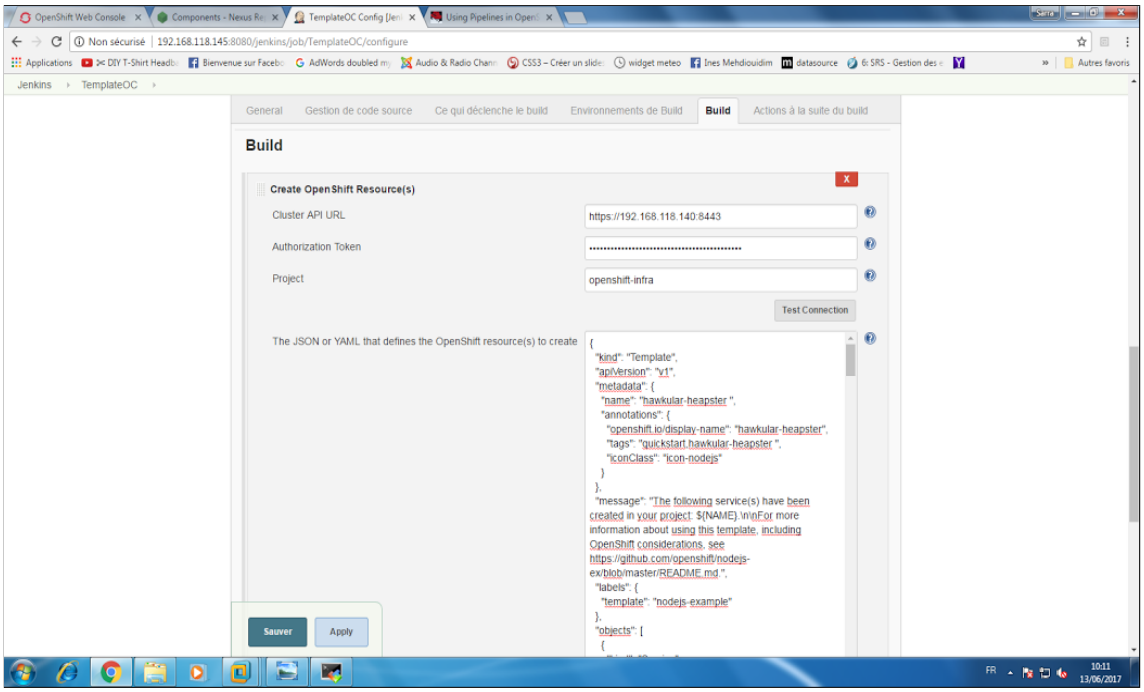


Figure 5.5: ICréation d'un template OpenShift avec Jenkins job.

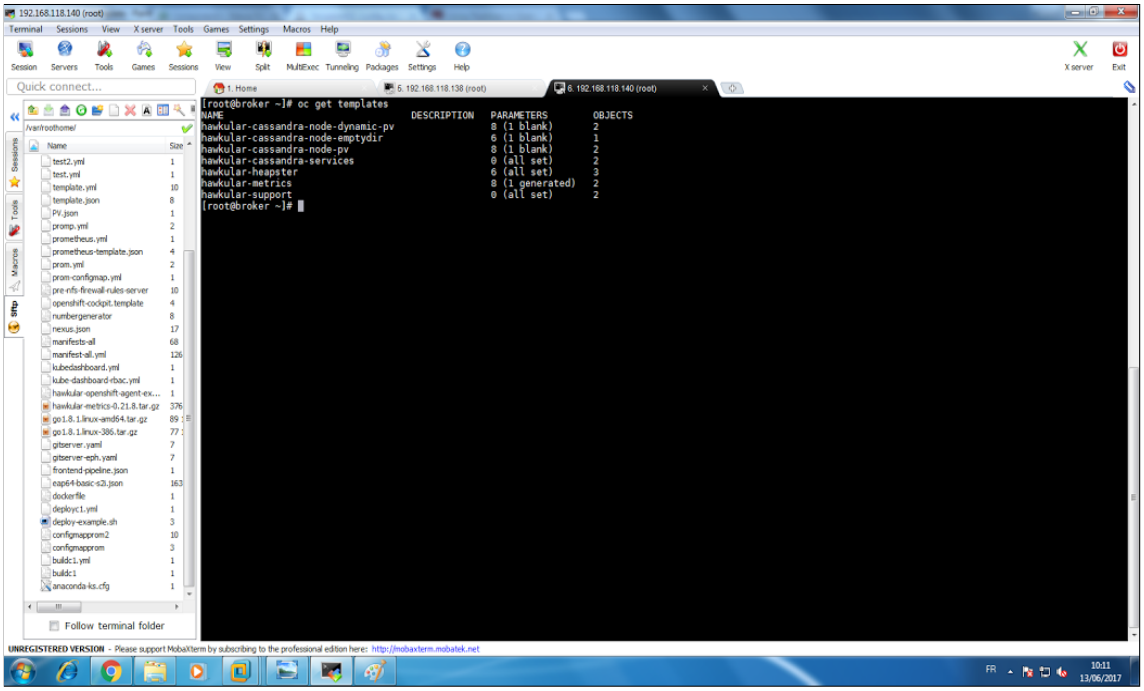


Figure 5.6: Création d'un template OpenShift avec Jenkins job.

5.3.3 Supervision

Le module de supervision permet de récupérer des métriques sur les composantes CPU, RAM, et pression entrée/sortie sur le réseau des pods du cluster. Il permet aussi de surveiller l'état des

conteneurs.

La réalisation de ce module a nécessité le déploiement de quatre éléments, Heapster pour la récupération des métriques, Hawkular pour leurs stockage, cassandra pour leurs persistances et enfin Hawkular OpenShift Agent pour pouvoir exécuter ce processus à travers tous les nœuds du cluster.

Pour la supervision de l'état des conteneurs le déploiement de kubernetes-dashboard a été nécessaire.

La figure 5.7 , montre l'usage CPU et RAM ainsi que la pression des entrées/sortie sur les conteneurs hawkular-cassandra et hawkular-metrics.

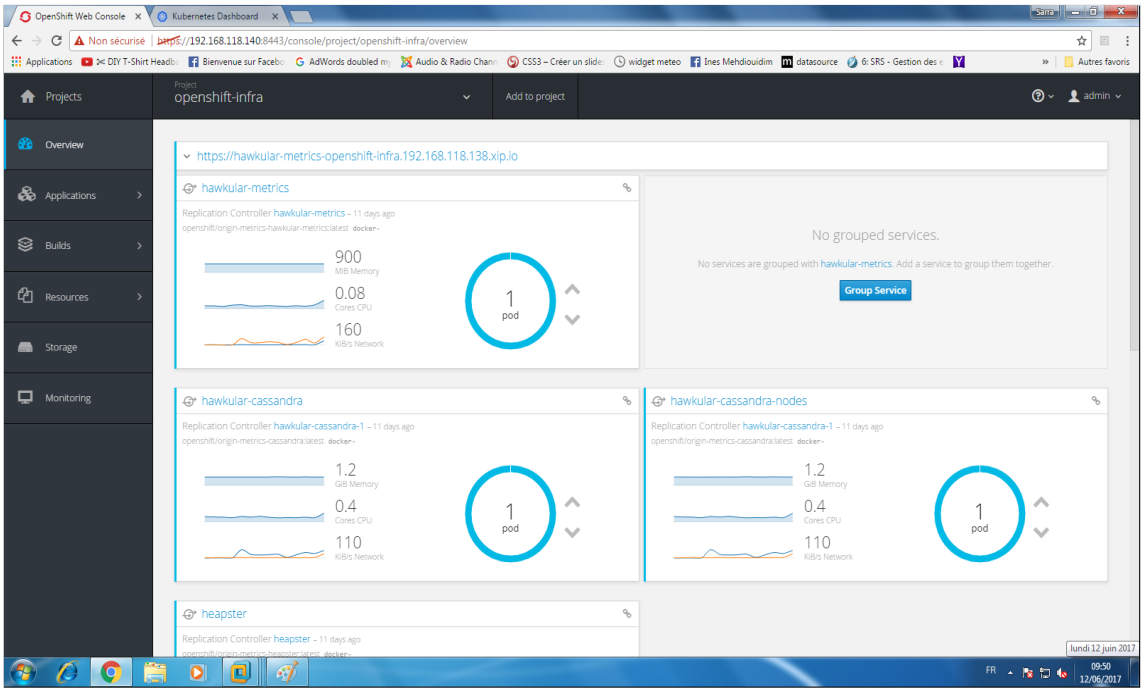


Figure 5.7: Affichage de métriques.

La figure 5.8 répond au cas d'utilisation « surveiller état des conteneurs », où nous pouvons voir le statut de chaque conteneur dans le projet "default" sur cet exemple.

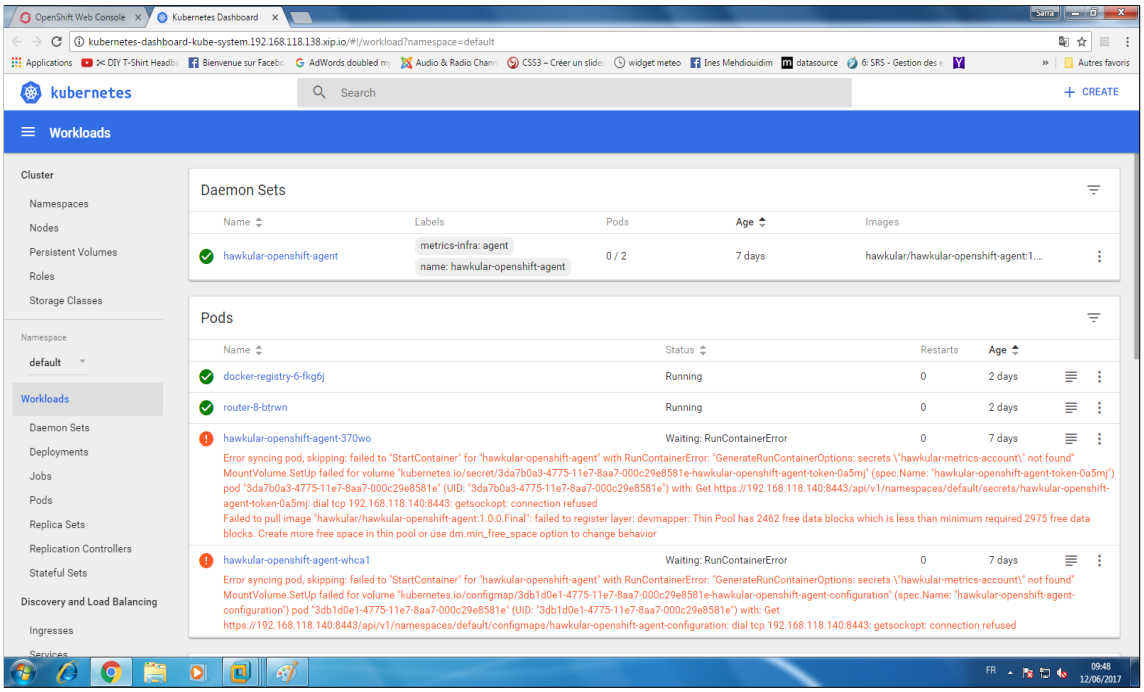


Figure 5.8: Affichage de l'état des conteneurs.

5.3.4 Pré-configuration

Dans cette partie nous avons effectués les configurations nécessaires pour la mise en place une chaîne d'intégration continue incluant les serveurs Git, Jenkins et Nexus. Cette chaîne a été mise en place suivant deux architectures : une à base de machines virtuelles et une à base de conteneurs Docker.

Nous avons aussi mis en place un cluster OpenShift grâce à l'outil de configuration d'ordinateurs Ansible.

5.3.4.1 Mise en place d'un cluster OpenShift

Le Cluster OpenShift a été mis en place grâce à l'outil de gestion d'ordinateurs « Ansible », pour ce faire, nous avons dû créer un fichier appelé « inventory file » qui contient la liste des nœuds cibles et un fichier « playbook » qui contient la liste des actions à faire sur ces noeuds.

Une fois le playbook lancé, Ansible se met à récupérer des informations sur les machines cibles et commence par la suite à effectuer les tâches mentionné dans le playbook.

La figure 5.9 présente un extrait des logs affiché par Ansible lors de lancement d'un playbook visant à déployer un cluster OpenShift.

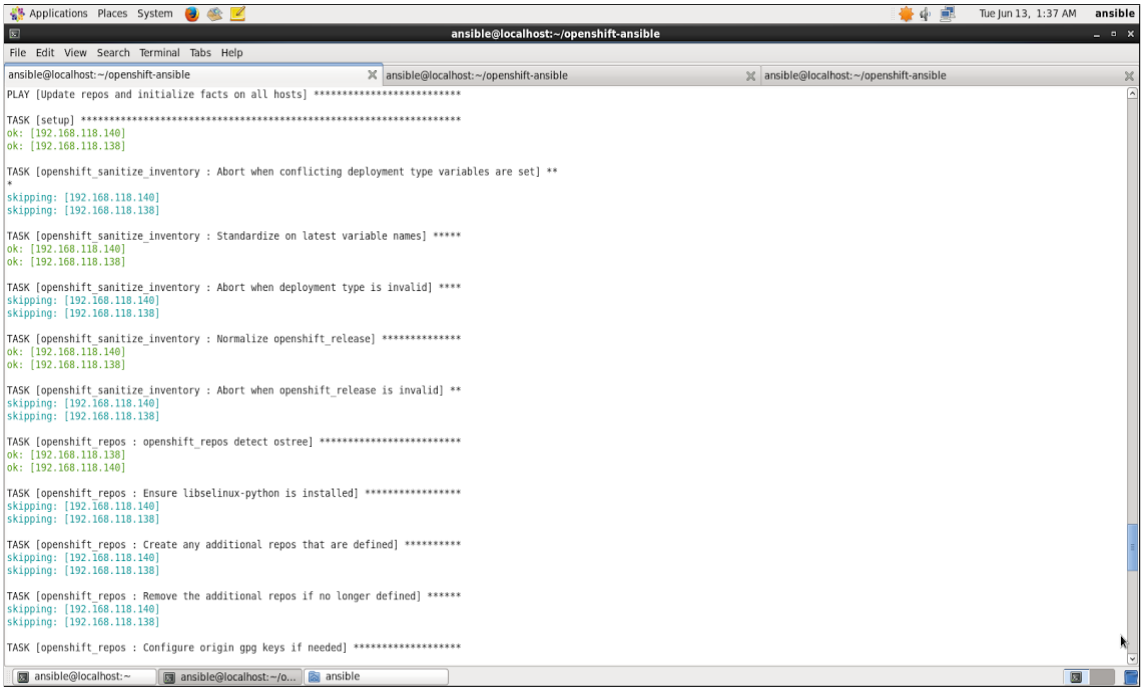


Figure 5.9: Logs Ansible.

L'imprime écran que représente la figure 5.10 , est la page d'accueil d'OpenShift contenant la liste des projets présent.

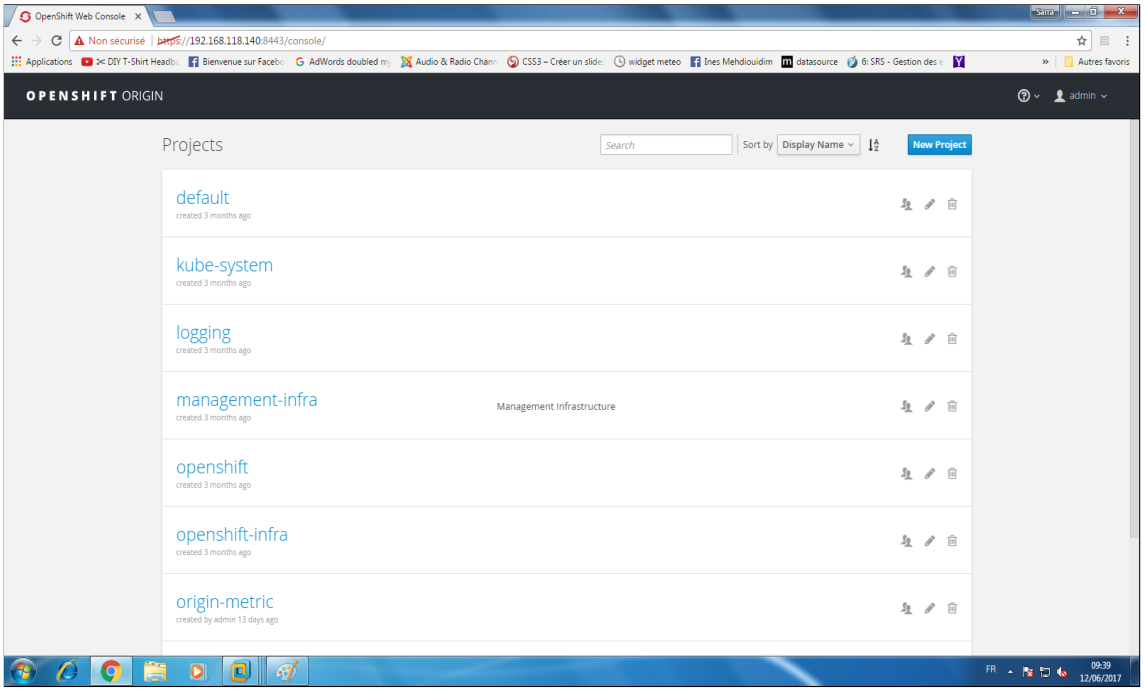


Figure 5.10: Console OpenShift.

L'imprime écran présent dans la figure 5.11, représente un extrait de la description de l'état

du cluster OpenShift après son déploiement avec Ansible.

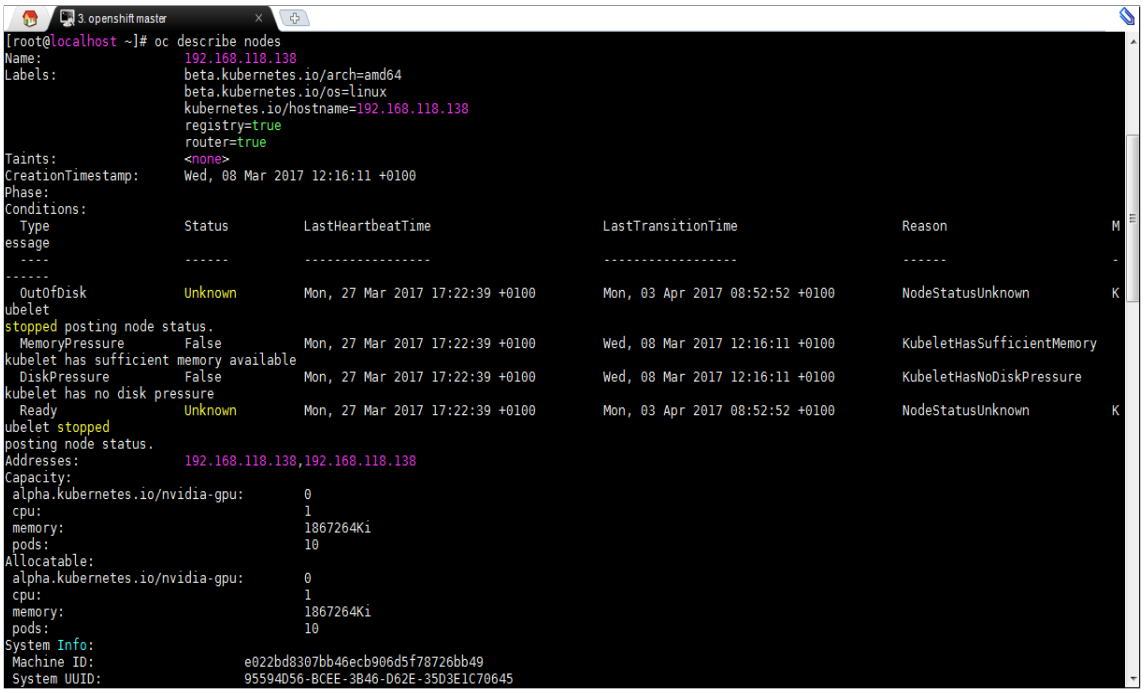


Figure 5.11: Extrait de la description de l'état du cluster OpenShift.

5.3.4.2 Mise en place d'un pipeline d'intégration continue

Comme nous l'avons déjà mentionné, la mise en place du pipeline d'intégration continu a été faîtes suivant deux architectures de déploiement. Nous allons commencer par expliquer la mise en place du serveur Git suivie de celle du serveur Jenkins pour conclure avec la mise en place du serveur Nexus, le tout, suivant les deux architectures de déploiement.

Installation Serveur Git : pour installer le serveur Git, nous avons procédé de deux manières, une à base de machine virtuelle et une à base de conteneur.

Installation Serveur Git à base de machines virtuelles : pour installer le serveur Git, il faut tout d'abord installer la commande git, ensuite il faut créer un dossier .ssh et attribuer les droits de lecture/écriture et exécution sur ce dossier à l'utilisateur de la session. Ensuite, il faut générer une clé publique qui sera copiée sur toutes les machines clientes du serveur Git. Il ne reste plus qu'à créer un dépôt git pour y déposer le code de l'application.

Installation Serveur Git à base de conteneurs : pour installer le serveur Git comme étant un conteneur sur OpenShift, il faut récupérer le template persistent de gitserver, sans la

persistance à chaque fois que le pod contenant le conteneur est fermé, le dépôt est réinitialisé à zéro. Ensuite, il faut donner aux conteneurs sur OpenShift le droit d'accès au conteneur Git pour pouvoir récupérer le code à partir de son dépôt.
Il faut aussi configurer les paramètres de connexion, c'est-à-dire, utilisateurs et mot de passe correspondant. Il est nécessaire de sécuriser Git et ceci sera fait avec le protocole TLS, en sécurisant la route d'accès au serveur, en configurant les certifications privé et en interdisant le clonage automatique à partir de son dépôt.

Installation Serveur Jenkins : pour installer le serveur Jenkins, nous avons procédé de deux manières, une à base de machine virtuelle et une à base de conteneur.

Installation Serveur Jenkins à base de machines virtuelles : l'installation de serveur Jenkins requiert tout d'abord, l'installation de Java et la configuration des variables d'environnement (java_home, jre_home et path), ensuite, nous procédons à l'installation du serveur apache tomcat que nous rendons exécutable en tant que script et au sein duquel nous déployons un fichier jenkins.war.

Installation Serveur Jenkins à base de conteneur : pour installer le serveur Jenkins comme étant un conteneur sur OpenShift, il suffit de récupérer le template jenkins persistant qu'OpenShift offre et de désactiver Selinux dans la machine OpenShift master.
La figure 5.12 est la page d'accueil de Jenkins, elle contient la liste des projets.

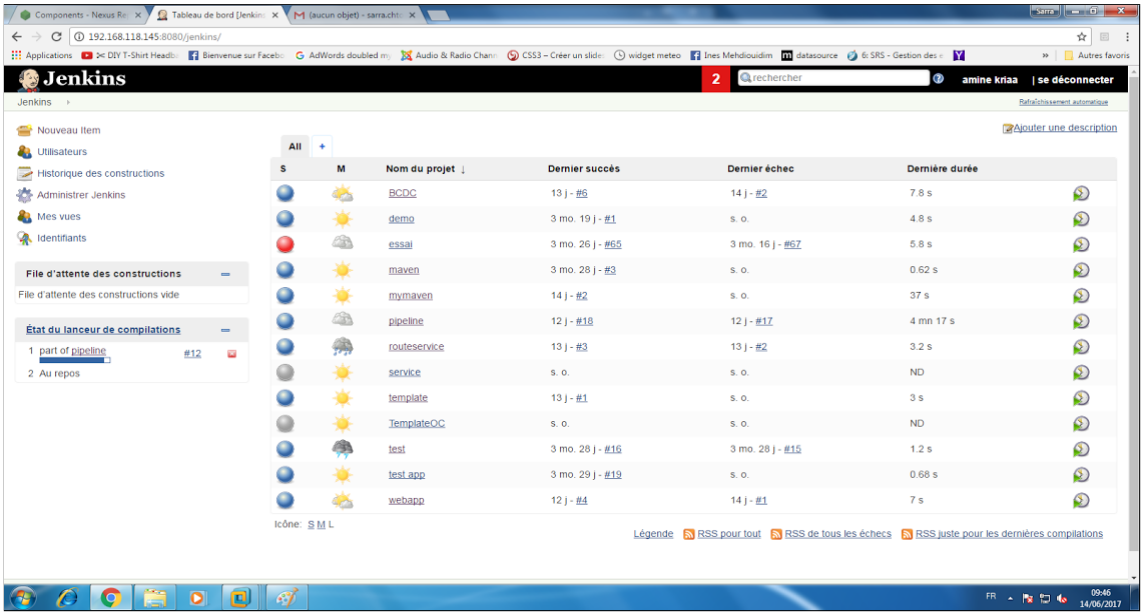


Figure 5.12: Liste des projets Jenkins.

Installation Serveur Nexus : pour installer le serveur Nexus, nous avons procédé de deux manières, une à base de machine virtuelle et une à base de conteneur.

Installation Serveur Nexus à base de machines virtuelles : l'installation d'un serveur Nexus nécessite l'installation d'openJdk sur la machine hôte, ensuite il faut modifier le fichier de configuration de Nexus pour attribuer au serveur un utilisateur de session non sudoer, car il est fortement déconseillé de le faire pour des raisons de sécurité. Il suffit ensuite de rendre l'exécution de Nexus en tant que service pour pouvoir le démarrer et l'arrêter avec des commandes.

Installation Serveur Nexus à base de conteneur : installer le serveur Nexus en tant que conteneur sur OpenShift requiert la récupération d'un template Nexus et la modification de son BuildConfig en y ajoutant un volume déjà existant sur OpenShift et ceci pour garantir la persistance. La figure 5.13 présente l'interface d'authentification du serveur Nexus.

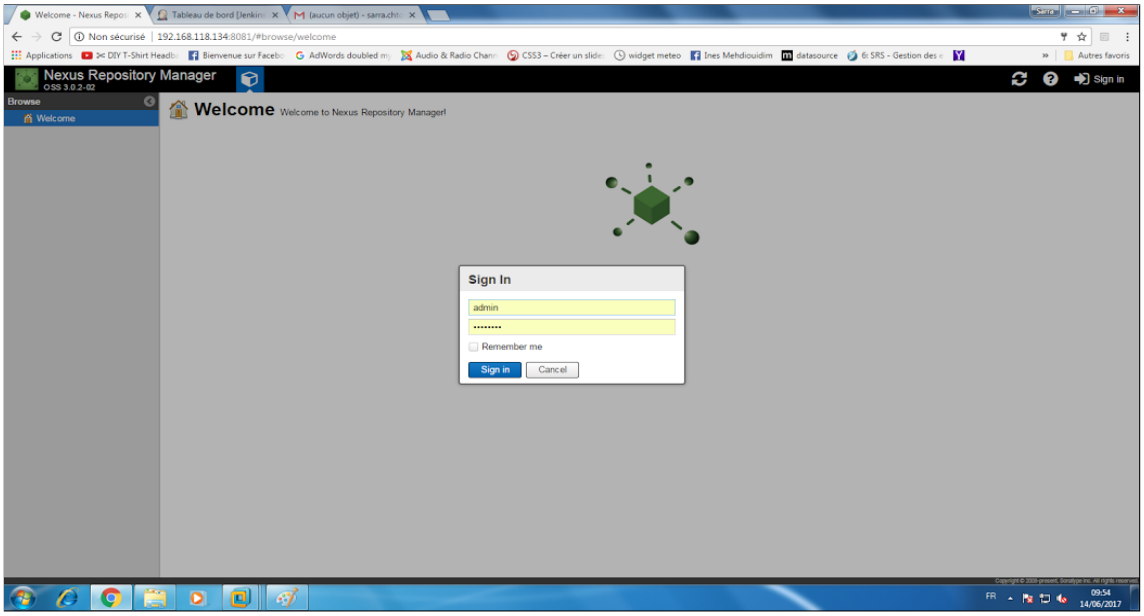


Figure 5.13: Interface d'authentification Nexus.

L'imprime écran exposé dans la figure 5.14 illustre une partie des éléments de la chaîne d'intégration continu, à savoir,les serveurs Git et Jenkins comme des conteneurs docker sur OpenShift.

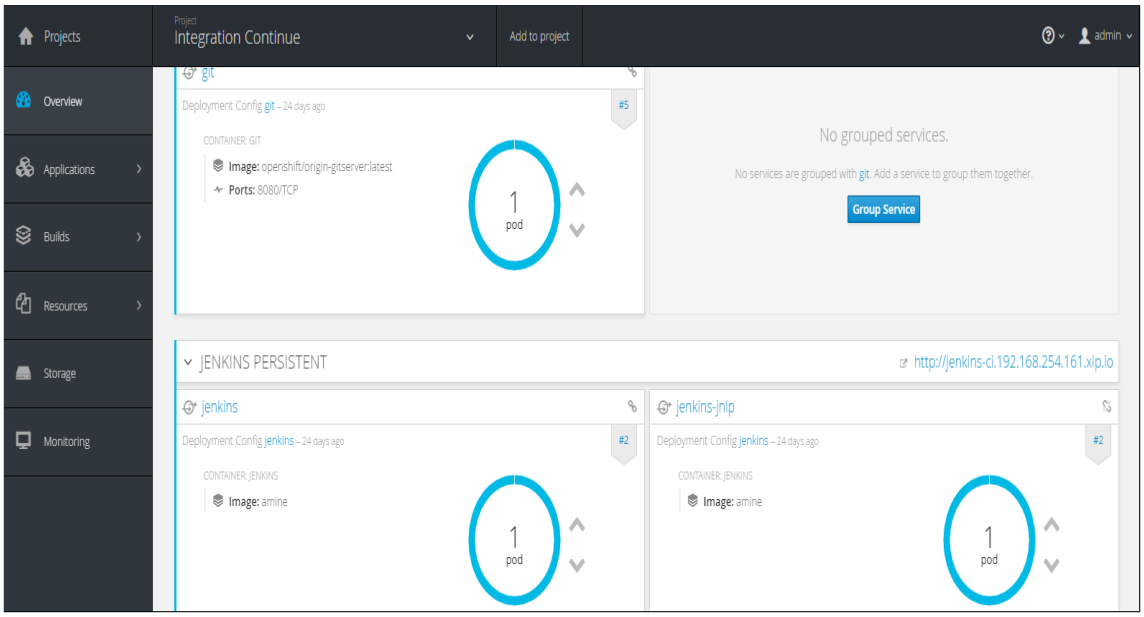


Figure 5.14: Pipeline intégration continue sur OpenShift

Conclusion

Ce dernier chapitre porte sur la réalisation de notre travail, il nous a permis de présenter l'environnement matériel et logiciel de notre projet, il nous a aussi permis de présenter les différentes technologies que nous avons utilisé pour l'aboutissement de ce travail, et enfin, nous avons exposés quelques interfaces graphiques relatives à notre projet.

Conclusion générale

Ce travail s'inscrit dans le cadre de projet de fin d'études, il a été réalisé au sein de la société Vermeg et consiste en la mise en place d'une chaîne d'intégration et de déploiement continus à base de docker accompagné d'un tableau de monitoring. Nous avons décomposé notre travail sur cinq modules, un module de d'intégration continue, un module de déploiement continu, un module de configuration, un module de monitoring et un module de pré-configuration.

Pour aboutir à ce travail, nous avons d'abord commencé par présenter le cadre général de notre projet qui inclut la présentation de l'organisme d'accueil, l'étude de l'existant et constatations, objectifs et contributions, et enfin l'application de la méthodologie adoptée. Ensuite, nous sommes passée à l'étude préalable qui a permis de définir les concepts clés pour notre projet, de comparer les différentes solutions techniques qui s'offrent à nous pour enfin faire synthèse de nos choix techniques. Puis, nous nous sommes intéressée à l'analyse et spécification, un chapitre qui a fait l'objet sur l'identification des acteurs, l'analyse des besoins et leurs représentation. Nous avons enchaîné avec l'architecture et conception détaillé qui a permis de présenter l'architecture générale de notre solution et décortiquer son aspect dynamique. Enfin, nous avons entamé la partie réalisation qui a porté sur la présentation de l'environnement de travail qu'il soit matériel ou logiciel, la présentation des différentes technologies utilisées pour réaliser notre travail, et pour finir l'exposition de quelques interfaces graphiques relatives à notre projet.

Grâce à cette expérience, nous avons pu, d'une part, découvrir de nouvelles technologies telle que docker , kubernetes et OpenShift et d'autres part acquérir de nouvelles compétences telle que l'administration système et le déploiement d'un cluster.

Par ailleurs, ce projet nous a aussi été bénéfique de point de vue personnel, il nous a permis d'appréhender le travail au sein d'une grande société et dans une hiérarchie professionnelle. Le travail de groupe, la gestion de projet de façon méthodique et la répartition des tâches et des efforts sont des qualités que nous avons acquises grâce à ce stage.

Nous tenons à présent à souligner quelques extensions intéressantes pour améliorer notre

projet. Nous estimons que l'ajout d'un serveur SonarQube pour la gestion de la qualité de code au niveau du pipeline d'intégration continu est souhaitable, aussi, nous proposons l'utilisation des indicateurs offert par SonarQube pour la génération d'autres mesure de qualité intéressantes pour les décideurs, tel que les violations qui sont les plus commise.
Nous pensons aussi, que superviser les « push » effectués sur Git pourra être utile étant donné que cela constitue un indicateur sur les heures où il est le plus sollicité, et donc où les équipes sont le plus stressé. Cet indicateur pourra signaler au décideur le déséquilibre de productivité qu'il peut y avoir au sein des équipes et de prendre des décisions en conséquence.

Pour conclure, nous souhaitons que ce travail apporte la satisfaction aux responsables de l'entreprise hôte « Vermeg » et aux membres du Jury.

Bibliographie

[1] VERMEG. (Jan. 2016). Vermeg. [Accès le 5-Mai-2017], adresse : https://www.vermeg.com/.

[2] thierry PIGOT. (Mai. 2011). Scrum-en-moins-de-10-minutes. [Accès le 13-Mai-2017], adresse : http://www.thierry-pigot.fr/scrum-en-moins-de-10-minutes/.

[3] communauté CATCHU. (Fev. 2011). La démarche projet 2tup. [Accès le 13-Mai-2017], adresse : http://www.catchu.fr/accelerateur/la-demarche-projet-2tup/.

[4] communauté UML-SYSML. (Jan. 2000). 2tup. [Accès le 13-Mai-2017], adresse : http://www.uml-sysml.org/images-du-site/2TUP.gif/view.

[5] A. SÉBASTIEN. (Fev. 2013). Intégration continue. [Accès le 13-Mai-2017], adresse : http://igm.univ-mlv.fr/~dr/XPOSE2012/Integration%20Continue/concept.html.

[6] communauté GITHUB. (Janv. 2015). Jenkins le guide complet. [Accès le 13-Mai-2017], adresse : http://jenkins-le-guide-complet.github.io/html/chapter-automated-deployment.html.

[7] communauté journal du NET. (Mai. 2016). Docker : ce qu'il faut savoir. [Accès le 3-Mars-2017], adresse : http://www.journaldunet.com/solutions/cloud-computing/1146290-cloud-pourquoi-docker-peut-tout-changer/.

[8] communauté le mag IT. (Jui. 2015). Cluster. [Accès le 04-Juin-2017], adresse : http://www.lemagit.fr/definition/Cluster.

[9] M. ROUSE. (Mars. 2016). What is kubernetes. [Accès le 04-Juin-2017], adresse : http://www.lemagit.fr/definition/Kubernetes.

[10] communauté SYNBIOZ. (Jan. 2010). Adopter un modèle de versionnement efficace avec git. [Accès le 20-fev-2017], adresse : https://www.synbioz.com/blog/git-adopter-un-modele-de-versionnement-efficaces.

[11] P. VENEZIA. (Nov. 2013). Review : puppet vs. chef vs. ansible vs. salt. [Accès le 5-Mars-2017], adresse : http://www.infoworld.com/article/2609482/data-center/data-center-review-puppet-vs-chef-vs-ansible-vs-salt.html?page=2.

[12] K. Boeckman. (Mars. 2016). Docker containers vs. virtual machines : what's the difference ? [Accès le 1-Mars-2017], adresse : https://newsroom.netapp.com/blogs/containers-vs-vms/.

[13] O. Dar. (Dec.2009). Openshift 3 : le paas privé avec docker. [Accès le 14-Juin-2017], adresse : https://dzone.com/articles/maven-repository-manager-nexus.

[14] communauté openvswitch. (Dec. 2016). Compare openshift vs. cloud foundry. [Accès le 22-Avr-2017], adresse : https://comparisons.financesonline.com/openshift-vs-cloud-foundry.

[15] O. community. (Jan. 2016). What is open vswitch ? [Accès le 3-Mars-2017], adresse : http://docs.openvswitch.org/en/latest/intro/what-is-ovs/.

[16] communauté rezo.net. (Sep. 2009). Je suis fan de yaml. [Accès le 12-Mai-2017], adresse : http://zzz.rezo.net/Je-suis-fan-de-YAML.html.

[17] B. Donaux. (Juin. 2015). Tuto docker - comprendre docker (partie1). [Accès le 12-Juin-2017], adresse : https://www.wanadev.fr/23-tuto-docker-comprendre-docker-partie1/.

[18] M. Rouse. (Nov. 2015). Machine virtuelle (vm). [Accès le 12-Juin-2017], adresse : http://www.lemagit.fr/definition/Machine-Virtuelle-VM.

[19] E. Devouge. (Juin . 2015). Openshift 3 : le paas privé avec docker. [Accès le 12-Juin-2017], adresse : http://blog.octo.com/openshift-3-le-paas-prive-avec-docker/.

ملخّص

من أجل تحسين التحكم في تطبيقاتها، منذ تحميل الكود إلى حين تسليم أجزاء التطبيق إلى الحريف، دأبت شركة فْرماقْ إلى وضع سلسلة ادماج و نشر كود مستمر مع التمكين من الإشراف عليها من خلال لوحة متابعة. في هذا الإطار و من خلال مشروع التخرج، طلب منا إنجاز هذا العمل الذي أتم بواسطة العديد من التكنولوجيات ألا و هي Kubernetes، OpenShift، Ansible، Git، Jenkins، Nexus، Docker.

كلمات مفاتيح : **إدماج مستمر، نشر مستمر، لوحة متابعة**، OpenShift، Docker.

Résumé

Afin de mieux gérer le cycle de vie de ses applications, depuis la déposition du code source dans un serveur de gestion de version, jusqu'à la livraison d'une release au client ; et afin de réduire le délai des livrables, Vermeg a opté pour la mise en place d'une chaîne d'intégration et de déploiement continus permettant l'automatisation du processus susmentionné. Tout ceci devra être supervisé et affiché dans un dashboard de monitoring. C'est dans ce cadre et en guise de projet de fin d'étude, que nous avons été appelé à mettre en œuvre ce travail. Ce projet a été réalisé avec les technologies Docker , Kubernetes OpenShift, Ansible, Git, Jenkins et Nexus.

Mots clés : intégration continue, déploiement continu, monitoring, Openshift, docker.

Abstract

In order to enhance the management of application's lifecycle from the source code deposition in a version management server to the release delivery, Vermeg chose to setup an integration and deployment pipeline supervised by a monitoring dashboard. Within this context and as end of study project, we brought this solution to light using several technologies such as Docker, Kubernetes OpenShift, Ansible, git, Jenkins and Nexus.

Keywords : integration pipeline, deployement pipeline, monitoring, Openshift, docker.

نهج بحيرة ملاران ـ ضفاف البحيرة ـ تونس الهاتف : 111 111 71 الفاكس : 222 222 71 البريد الالكتروني : contact@company.com
Rue du Lac Malaren, Les Berges du Lac 1053 Tunis **Tél** : 71 111 111 **Fax** : 71 222 222 **Email** : contact@company.com

2 نهج أبو الريحان البارونى 2080 أريانة الهاتف : 164 706 71 الفاكس : 698 706 71 البريد الالكتروني : isi@isim.rnu.tn
2, Abou Raihane Bayrouni 2080 l'Ariana **Tél** : 71 706 164 **Fax** : 71 706 698 **Email** : isi@isim.rnu.tn